

ETAS RTA-BSW v12.7.0



RTA-RipsPlugins Stack Reference Guide

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-RipsPlugins Stack Reference Guide R01 EN – 05.2025

Contents

1	RTE Implementation Plugin Support	5
1.1	Introduction	5
1.2	Supported Features	5
1.3	Extensions	6
1.4	Limitations	6
1.5	Exclusions	6
1.6	Deviations	6
1.7	API Definition	6
1.8	Configuration Advice	6
1.9	Integration Advice	7
1.10	Hardware Requirements	7
2	RTE Implementation Plugin Support for C/S Communication with Transformers	8
2.1	Introduction	8
2.2	Supported Features	8
2.3	Extensions	8
2.4	Limitations	13
2.5	Exclusions	13
2.6	Deviations	13
2.7	API Definition	15
2.8	Configuration Advice	15
2.9	Integration Advice	17
2.10	Hardware Requirements	24
3	RTE Implementation Plugin Support for S/R Communication with Transformers	25
3.1	Introduction	25
3.2	Supported Features	25
3.3	Extensions	26
3.4	Limitations	28
3.5	Exclusions	28
3.6	Deviations	29
3.7	API Definition	30
3.8	Configuration Advice	30
3.9	Integration Advice	31
3.10	Hardware Requirements	38

4	RTE Implementation Plugin Support for Signal to Service Translation	39
4.1	Introduction	39
4.2	Supported Features	40
4.3	Extensions	41
4.4	Limitations	43
4.5	Exclusions	43
4.6	Deviations	44
4.7	API Definition	45
4.8	Configuration Advice	46
4.9	Integration Advice	49
4.10	Hardware Requirements	50
5	RTE Implementation Plugin Support for Safety C/S Communication	51
5.1	Introduction	51
5.2	Supported Features	52
5.3	Extensions	53
5.4	Limitations	55
5.5	Exclusions	55
5.6	Deviations	55
5.7	API Definition	57
5.8	Configuration Advice	57
5.9	Integration Advice	57
5.10	Hardware Requirements	66
6	Glossary	67

1 RTE Implementation Plugin Support

Module name	Rte_Rips
Package	RTA-RIPSPLUGINS
AUTOSAR Module ID	n/a
AUTOSAR Version	22-11

1.1 Introduction

RTE Implementation PluginS (RIPS) enables a modularized implementation of the RTE by providing a standardized interface through which to connect external plugins.

This interfacing to RTE is based on “global” instances that are related to communication paths taken from FlatInstanceDescriptors. The supplied ARXML model (e.g. software component descriptions, FlatMap etc) defines the contract between the generated RTE code and the plugin. This contract includes static interfaces and some behavioral rules.

The plugin interfaces are designed in such a way that the RIPS Generator requires only a sub-set of the information that the RTE Generator needs to create the full interfacing for software components. The correct generation of the AUTOSAR interfaces to the software components, the schedule manager interfaces to basic software modules, and the interfaces to the communication services and transformers remain in the scope of the RTE Generator, while the plugin interfaces focus on the access to global communication paths.

This allows for the implementation of plugins with additional optimization for a particular domain of application, without the need to re-introduce the full complexity of the RTE Generator. The RTE Generator can provide a generic interface instead of implementing numerous user extensions.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

1.2 Supported Features

1.2.1 Memcpy Routine

The Rte_Rips module provides the Rte_Rips_Memcpy routine for an optimized copy of complex data types as arrays/structs.

1.2.2 Tick Interval Time

Tick interval time is supported and will require configuring in the RIPS configuration file Rte_Rips_Cfg.h.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

- No known extension parameters are defined in this version.

1.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

1.4 Limitations

There are currently no known limitations in the Rte_Rips module.

1.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

1.6 Deviations

1.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

- There are no known deviations in this version.

1.7 API Definition

List of supported API

Rte_Rips_MemCpy	Implements memcpy with the same parameters as C90-memcpy but copies 8/16/32 bit data based on the source and destination alignment.
-----------------	---

1.8 Configuration Advice

There is no specific configuration advice for the Rte_Rips module.

1.9 Integration Advice

1.9.1 Init Functions

No specific handling necessary.

1.9.2 Delnit Functions

No specific handling necessary.

1.9.3 Scheduled Functions

There are no scheduled functions.

1.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rte_Rips module does not use any exclusive OS resources.

1.10 Hardware Requirements

Table 1.3.Rte_Rips Hardware Requirements

Support	Usage
FLASH	88 bytes
RAM	0 bytes

Usage calculated based on a sample configuration.

2 RTE Implementation Plugin Support for C/S Communication with Transformers

Module name	Rte_Rips_CSXfrm
Package	RTA-RIPSPLUGINS
AUTOSAR Module ID	n/a
AUTOSAR Version	22-11

2.1 Introduction

The RIPS for C/S Communication with Transformers module (Rips_CSXfrm) enables SW-Cs allocated on a specific ECU instance to request services, via SomelpXf/E2EXf, and (Ld)Com, from other SW-Cs allocated on a different ECU instance.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

2.2 Supported Features

Rips_CSXfrm enables SW-Cs allocated on a specific ECU instance to request services from other SW-Cs allocated on a different ECU instance.

1. The Client triggers the RTE_Call to transmit data, which RTE redirects to Rte_Rips (via Rte_Rips_Invoke) which is implemented by Rips_CSXfrm.
2. The buffer handling and the serialization of the argument is done by SomelpXf/E2EXf.
3. After a successful serialization of the argument, data is sent to the Server via (Ld)Com.
4. The Server receives and deserializes the data, and informs the Client of the availability of the result via a notification callback.
5. The Client redirects the Rte_Rips_ReturnResult API via the Rte_Result API.
7. SomelpXf is responsible for the deserialization of the data and ensures that it is correctly passed to the SW-C.
8. Rte_Rips_CSXfrm relies on Os functionalities with regard to Timeout Handling via OsAlarm response. OsCounter.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 Rte_Rips_CSXfrm

Short Name	Rte_Rips_CSXfrm
Description	Configuration of the RIPS module to support Inter-ECU C/S Communication with Transformers.
Multiplicity	0..1
Default Value	-
Type	ECUC-MODULE-DEF

2.3.1.2 Rte_Rips_CSXfrm/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerServerResponseTimeout

Short Name	RteRipsInvocationHandlerServerResponseTimeout
Description	Specifies the maximum time of waiting for results from Server Runnable. Unit is seconds.
Multiplicity	0..1
Value Range	0..3600
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

2.3.1.3 Rte_Rips_CSXfrm/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerSupportsMultipleServerActivation

Short Name	RteRipsInvocationHandlerSupportsMultipleServerActivation
Description	Denotes if or if not the InvocationHandler supports the multiple activation of server runnable.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.4 Rte_Rips_CSXfrm/RteRipsPluginFillFlushRoutine/RteRipsPluginFillFlushRoutineSymbol

Short Name	RteRipsPluginFillFlushRoutineSymbol
Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

2.3.1.5 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginCallback-

MemMapPolicyRtePluginCallbackMemMapPolicy

Short Name	RtePluginCallbackMemMapPolicy
Description	Enable Generation of Callbacks based on ComSignals/LdComIPdus or SW-Cs
Multiplicity	1..1
Default Value	RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED
Enumeration Literals	RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED, RTE_RIPS_SIGNAL_BASED_MEMMAP_DISABLED
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.6 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginGenerateQueueFullErrorHook

Short Name	RtePluginGenerateQueueFullErrorHook
Description	Denotes if or if not the plug-in supports the Generation of Callout function in Server Queue Full Error Scenario.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.7 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginNoNestLockSupportEnable

Short Name	RtePluginNoNestLockSupportEnable
Description	Denotes if or if not the plug-in supports the Generation of NoNest Lock Functions.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.8 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsAutonomousError

Short Name	RtePluginSupportsAutonomousError
Description	Denotes if or if not the plug-in supports the Autonomous error response.
Multiplicity	1..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.9 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsInvokeReturnResult

Short Name	RtePluginSupportsInvokeReturnResult
Description	Denotes if or if not the plug-in supports the Rte_Rips_Invoke/ReturnResult APIs for (a)synchronous C/S Communication.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.10 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsLdComIF

Short Name	RtePluginSupportsLdComIF
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_IF.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.11 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsLdComTP

Short Name	RtePluginSupportsLdComTP
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_TP.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.12 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsSyncCommWithoutTimeout

Short Name	RtePluginSupportsSyncCommWithoutTimeout
Description	Denotes if or if not the plug-in shall support synchronous C/S Communication without Timeout.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.13 Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsTimeoutHandling

Short Name	RtePluginSupportsTimeoutHandling
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_TP.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.14 Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsTimeOutHandlingProps

Short Name	RteRipsTimeOutHandlingProps
Description	A list of OsTasks (one per OsApplication/EcucPartition) which will be used to invoke the MainFunctions generated by the RTE to perform timeout w.r.t async/sync C/S communication.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.15 Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsTimeOutHandlingProps/RteRipsTimeOutHandlingOsTaskRef

Short Name	RteRipsTimeOutHandlingOsTaskRef
Description	A reference to an OsTask that will be used to handle timeout for all clients allocated to the EcucPartition resp. OsApplication containing the referenced OsTask.
Multiplicity	0..*
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.16 Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsTimeOutHandlingProps/RteRipsTimeOutImplementation

Short Name	RteRipsTimeOutImplementation
Description	Denotes if or if not the Timeout Handle used instead of OsAlarm functions in the Generation.
Multiplicity	0..1
Default Value	OSALARMCBK_APPROACH

Enumeration Literals	OSALARMCBK_APPROACH, MAINFUNCTION_APPROACH
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

2.4 Limitations

- Synchronous C/S communication via Data Transformation through LdCom_TP is not supported.
- Synchronous C/S Communication with no Arguments is not supported.
- The Transformer Hard Error event is not yet handled.
- In the case of FanIn/FanOut, metadata is supported only when all of the signals have metadata configured.
- In the case of multiple LdComTP clients, a unique client ID must be configured for each.

2.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Rte_Rips_CSXfrm/RteRipsPluginFillFlushRoutine
- Rte_Rips_CSXfrm/RteRipsPluginFillFlushRoutine/RteRipsModeDisabling-Handling
- Rte_Rips_CSXfrm/RteRipsPluginFillFlushRoutine/RteRipsOsSchedulePoint
- Rte_Rips_CSXfrm/RteRipsPluginProps/RtePluginSupportsIReadIWrite
- Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsPluginCommunicationScope
- Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsPluginCommunicationType-Support
- Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsPluginCommunicationType-Support/RteRipsPluginCommunicationType

2.6 Deviations

2.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.1.1 Rte_Rips_CSXfrm/RteRipsInvocationHandler

[RTA-BSW] Parent Container	Rte_Rips_CSXfrm/RteRipsInvocationHandler
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler
[ADDED] Multiplicity	0..*

2.6.1.2 Rte_Rips_CSXfrm/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc

[RTA-BSW] Parent Container	Rte_Rips_CSXfrm/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[RTA-BSW] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables.
[AUTOSAR] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables and triggered runnables via a transformer.

2.6.1.3 Rte_Rips_CSXfrm/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol

[RTA-BSW] Parent Container	Rte_Rips_CSXfrm/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
[RTA-BSW] Description	C-Symbol of the Rte_Rips_InvocationHandler function.
[AUTOSAR] Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.

2.6.1.4 Rte_Rips_CSXfrm/RteRipsPluginProps

[RTA-BSW] Parent Container	Rte_Rips_CSXfrm/RteRipsPluginProps
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps
[REMOVED] Introduction	The shortName of the container defines the name of the RTE Implementation Plug-in used for the API infixes.

2.6.1.5 Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy

[RTA-BSW] Parent Container	Rte_Rips_CSXfrm/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[RTA-BSW] Description	Globally enables or disables the support for Rte Implementation Plug-Ins (RIPS) Tags: atp.Status=draft
[AUTOSAR] Description	Globally enables or disables the support for Rte Implementation Plug-Ins (RIPS)

2.7 API Definition

2.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

2.8.1 Configuration Containers

The following two containers need to be configured for the Rte_Rips_CSXfrm module.

RteRipsPluginProps

This container defines the plugin properties, and it is referenced by the RTE and Flat Instance Descriptor. None of the parameters in this container are currently used for code generation, and they should be configured with their default values as shown here:

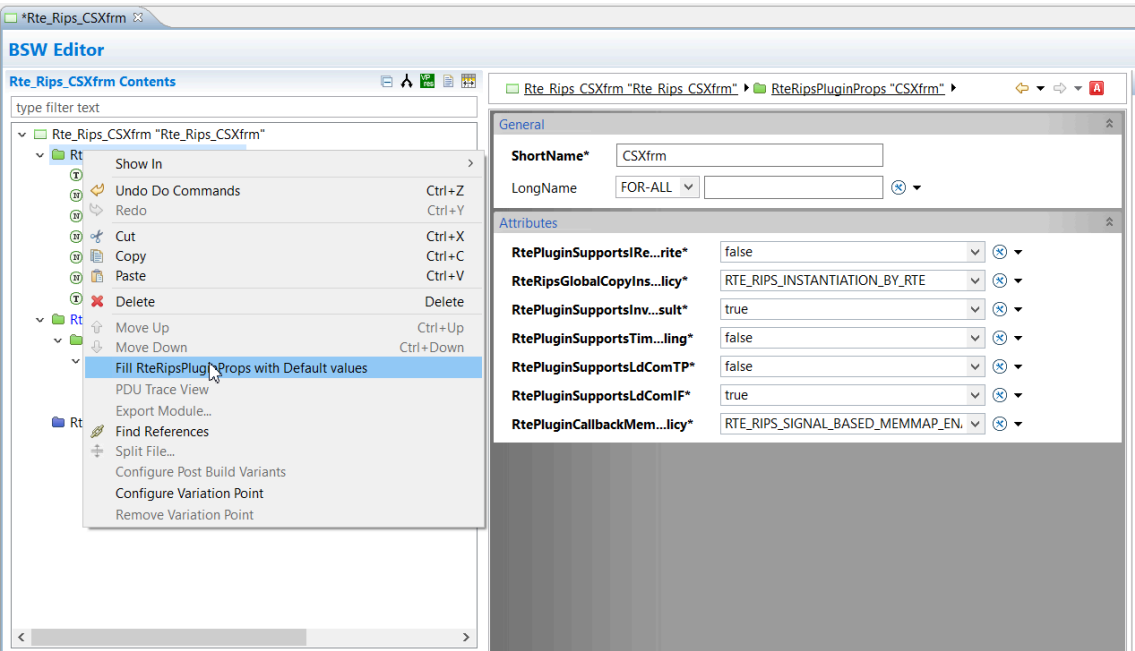


Figure 2.1.RteRipsPluginProps

RteRipsInvocationHandler

This container defines the Invocation Handler that will process service requests from other ECUs. This function will be generated by Rte_Rips_CSXfrm and scheduled through RTE by the configured Operation of Software Component that will serve for this request. See the Integration Advice for more details.

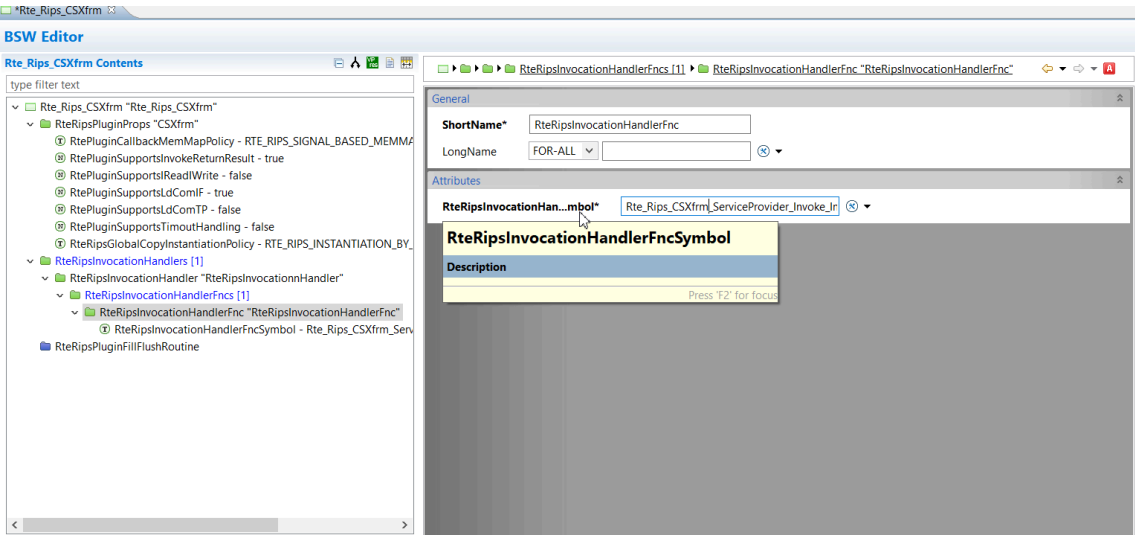


Figure 2.2.RteRipsInvocationHandler

2.8.2 General

For inter-ECU client-server communication you should always specify a timeout(`RteRipsInvocationHandlerServerResponseTimeout`) >0 when synchronous server calls are used, otherwise the client would wait indefinitely for the server response.

2.9 Integration Advice

2.9.1 Init Functions

`Rte_Rips_CSXfrm` is initialised by `Rte` in `Rte_Start`.

2.9.2 Delnit Functions

No shutdown is required for `Rte_Rips_CSXfrm`, but it is de-initialized by `Rte` in `Rte_Stop`.

2.9.3 Scheduled Functions

There are no scheduled functions.

2.9.4 Integration Use Case

The following sequence diagram illustrates a simple use case where an ECU can request services from other ECUs. It also highlights the dependencies between each module and `Rte_Rips_CSXfrm`.

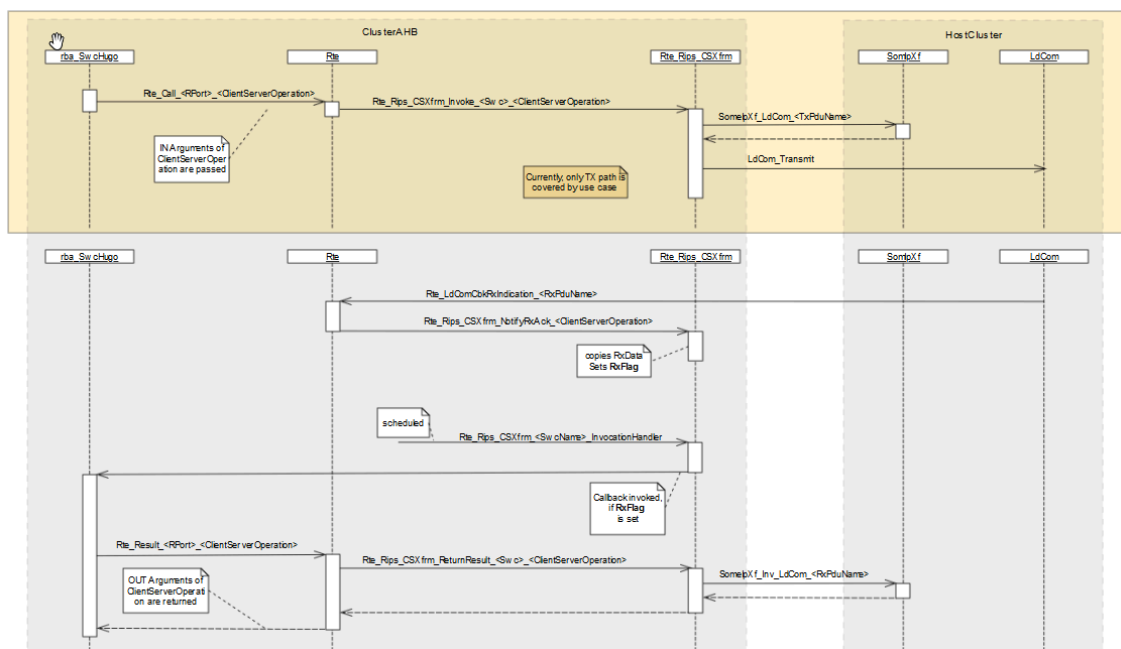


Figure 2.3.RIPS Sequence Diagram

As described in the Introduction section for the `Rte_Rips` module, RIPS is used to

create plugin features for RTE in order to make RTE software more modular. For this reason, even though it is considered as a module from BSW and generated by CodeGen, some additional configuration is required for the following:

- Application Software Component
- Inter ECU Communication for Client Server
- ECU Extract for Rips Property
- RTE
- OS
- SWC To ECU Mapping with Partition

2.9.4.1 Application Software Component

Rte_Rips_CSXfrm needs to know which A-SWC will be requesting or requiring services. This A-SWC will be fulfilled with the following:

- User defined Interfaces.
- An RPort is used to request a service from other ECUs.
- A Pport is used to serve service requests from other ECUs.
- A Runnable Entity is used to trigger the service request.
- A Runnable Entity is used to serve service requests from other ECUs.

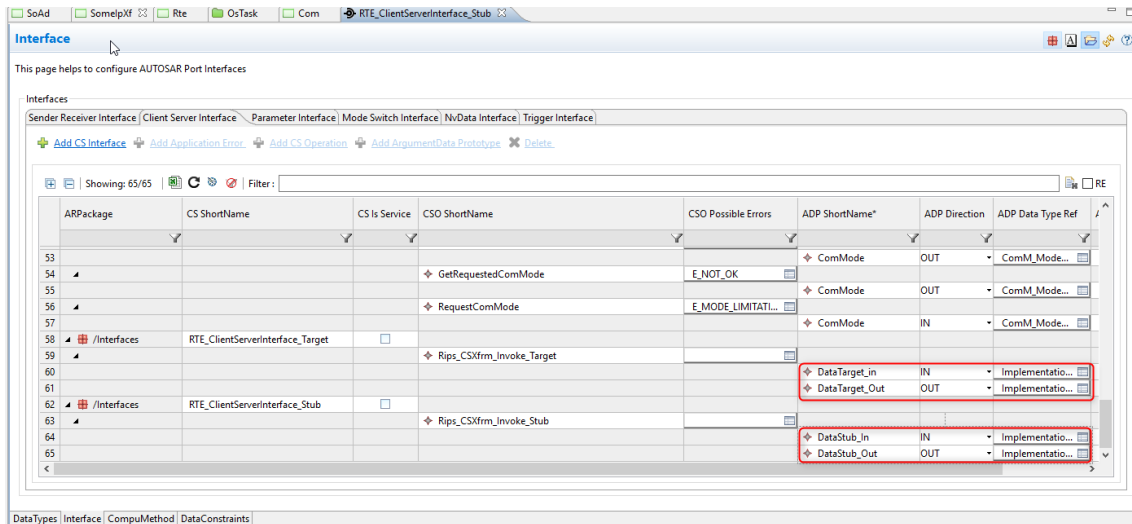


Figure 2.4. User Defined Interface

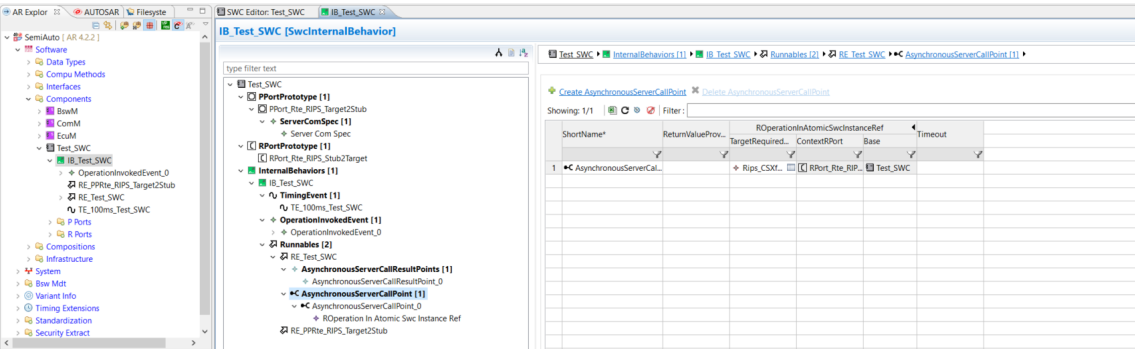


Figure 2.5.Application SWC

Note: Two call points for Rte_Call and Rte_Result are required if the ECU needs to request a service from another ECU.

2.9.4.2 Inter ECU communication for CS

In this example, an ECU will request a service from another ECU, so it is necessary to configure the relevant port as part of the Inter ECU Client Server communication.

Client Server Signal Mapping

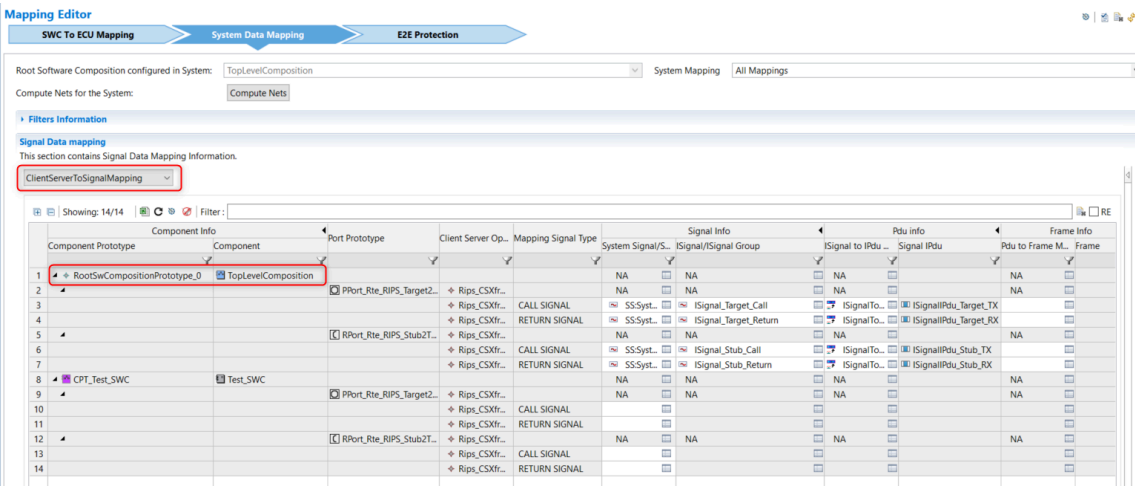


Figure 2.6.Client Server Signal Mapping

ECU Extraction

From the ECU Extraction for Flatview, delegation connectors are required for each port.

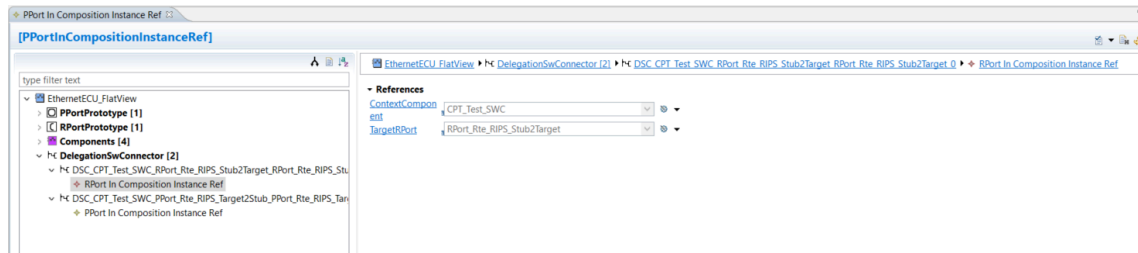


Figure 2.7. Configuration of Delegation Port

From the ECU Extraction for FlatMap, a Flat Instance Descriptor is required for each port. Note that this FlatMap for RIPS is only supported from a higher AR version, so you will first need to update the AR version before doing the configuration.

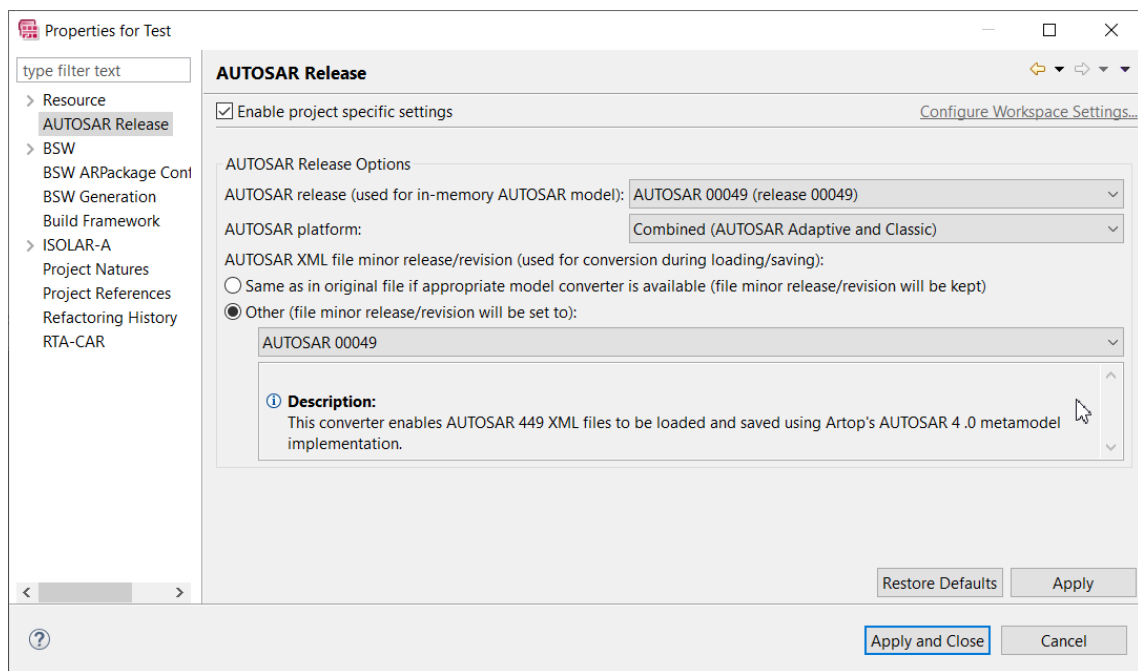


Figure 2.8. re-config AR version for RTA_CAR project

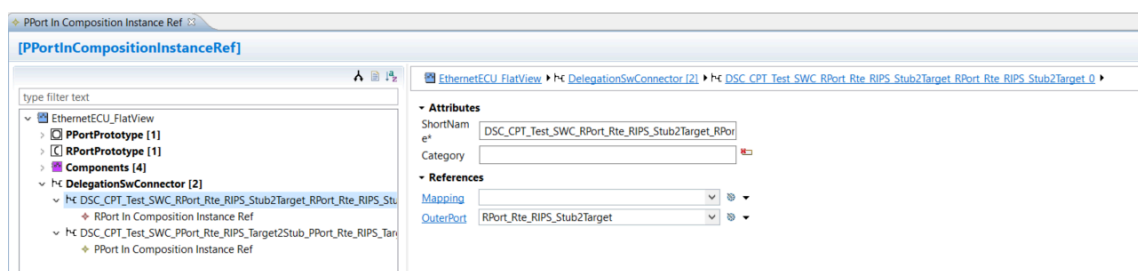


Figure 2.9. Flat Instance Descriptor

2.9.4.3 (Ld)Com Notification

When (Ld)Com receives an RX ComSignal, it should notify RTE with an API notification as `Rte_COMCbk_ComSignal_<RX_Signal>`, so in the Com configuration we need to configure notification for 2 signals as follows:

- Return/Response message from another ECU
- Call/Service request from another ECU

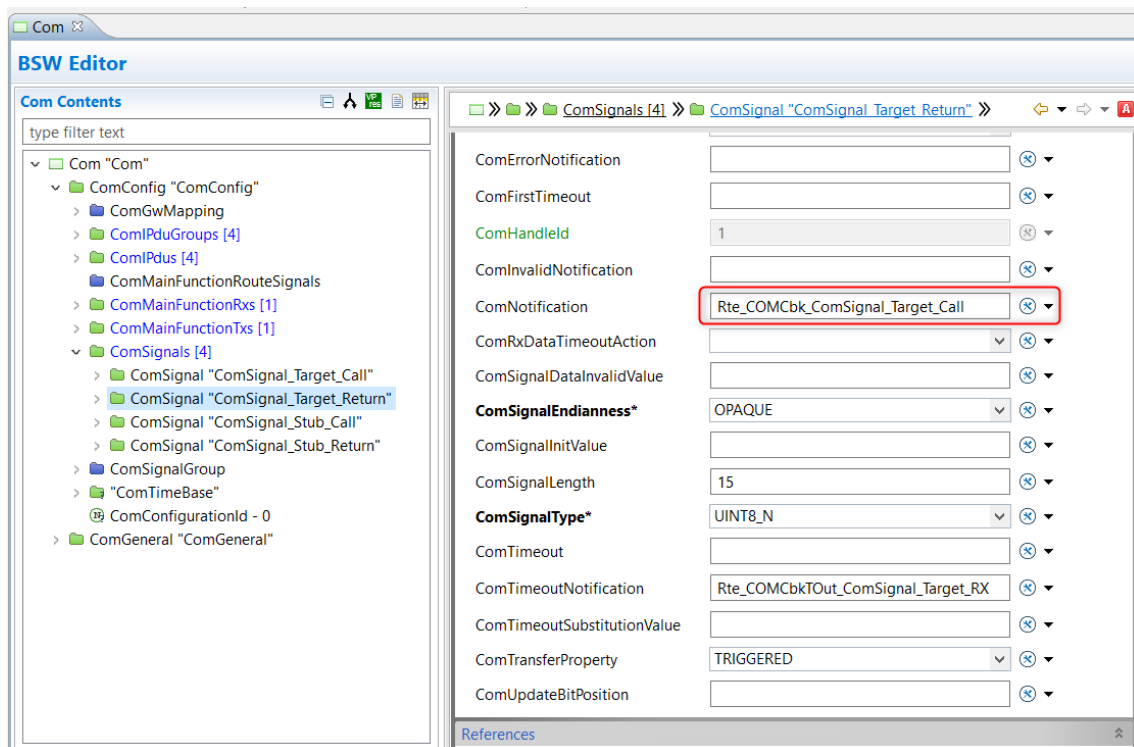


Figure 2.10.Com Configuration

Important Note: $\text{ComSignalLength} = \text{Total length of DataIN/OUT} + \text{SomeIpXf header length (8bytes)}$, where DataIN/OUT is the size of the user's data (defined as arguments of service requests).

2.9.4.4 RTE configuration

In the RTE configuration we need to set RIPS ON, and then map the Runnable Entity that will serve for service requests with the Invocation Handler from `Rte_Rips_CSXfrm`.

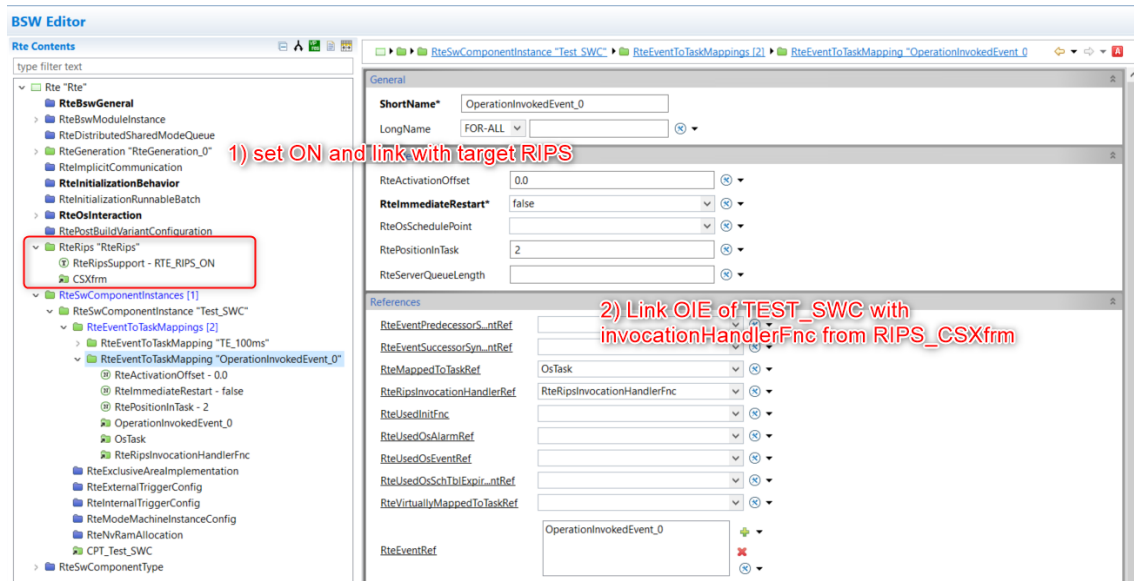


Figure 2.11.RTE Configuration

2.9.4.5 OS configuration

The OS needs to be configured with various mandatory elements including an OsApplication, a Partition and the relevant OsTasks into which the Runnables will be assigned.

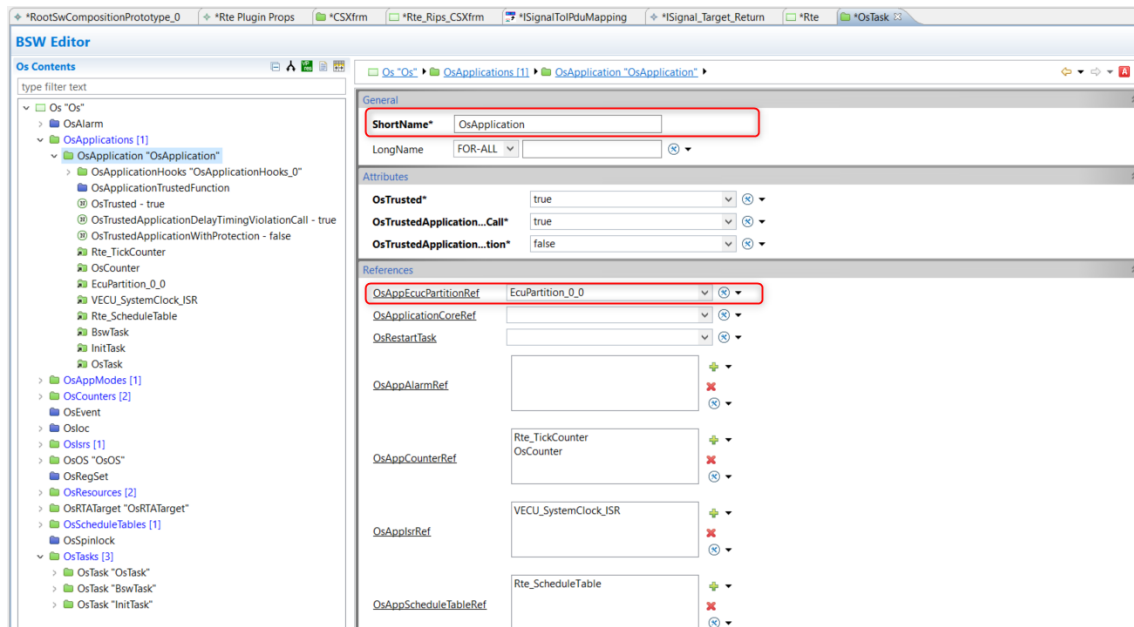


Figure 2.12.OS Configuration

For systems relying on OS with SC>SC1, support of OsAlarmCallbacks should be explicitly enabled.

2.9.4.6 SWC To ECU Mapping with Partition

All component prototypes mapped into the target ECU need to be mapped to a Partition, even in a single core environment.

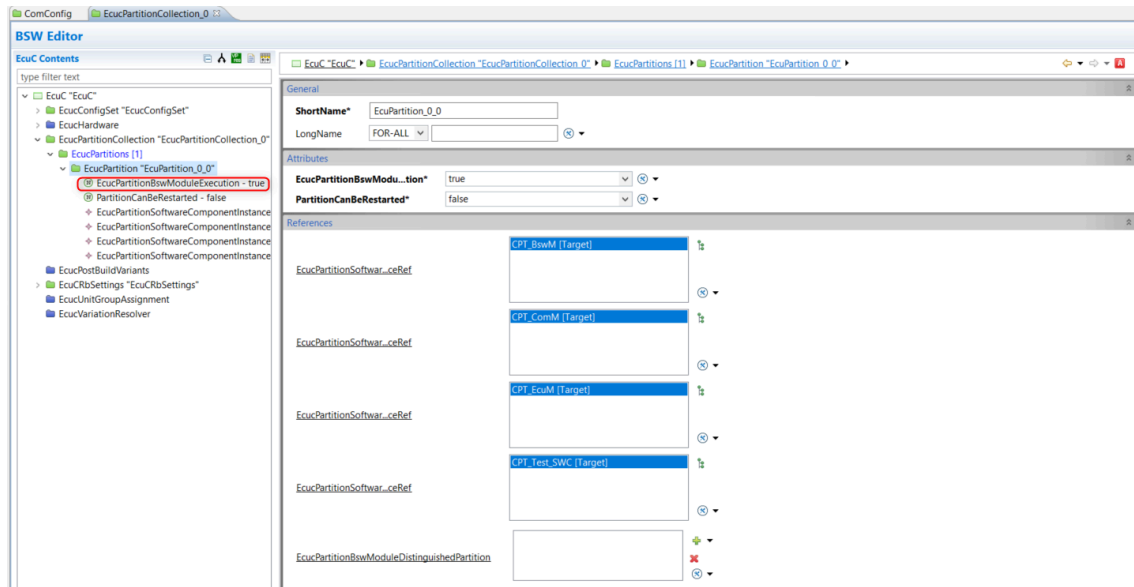


Figure 2.13. Ecu Enable Partition

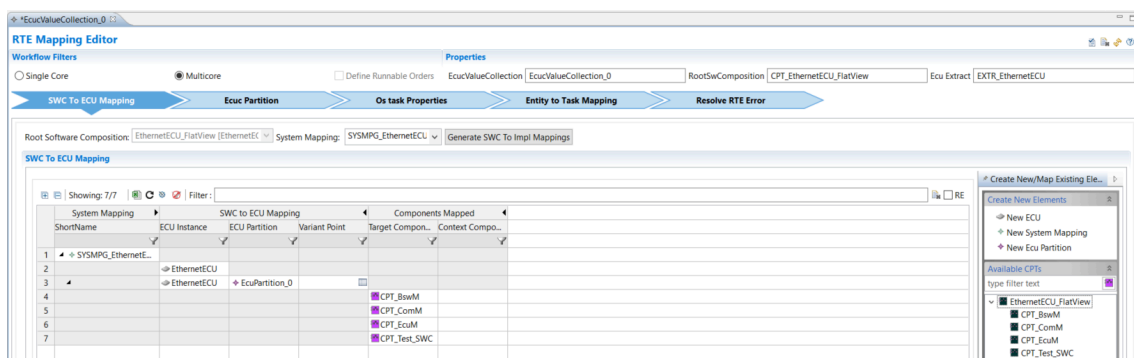


Figure 2.14. SWC to ECU Mapping with Partition

2.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rte_Rips_CSXfrm module does not use any exclusive OS resources.

2.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

3 RTE Implementation Plugin Support for S/R Communication with Transformers

Module name	Rte_Rips_SRCom
Package	RTA-RIPSPLUGINS
AUTOSAR Module ID	n/a
AUTOSAR Version	22-11

3.1 Introduction

The RIPS for S/R Communication with Transformers module (Rips_SRCom) enables SW-Cs allocated on a specific ECU instance to transmit/receive data via Transformers (SomelpXf, E2EXf) and LdCom (LDCOM_TP only) to/from other SW-Cs allocated on a different ECU instance. Transmission/Reception of byte arrays without Data Transformation is also supported.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

3.2 Supported Features

Rips_SRCom enables SW-Cs allocated on a specific ECU instance to request services from other SW-Cs allocated on a different ECU instance.

On the Transmission side, an SW-C invokes the Rte_Rips_Write via the Rte_Write API to perform data transformation (SomelpXf, E2EXf) and send the transformed data to the receiving SW-C via LdCom/Com. Transmission of byte arrays without Data Transformation is also supported.

On the Reception side, the transformed data is received by the Rte_Rips_SRCom from LdCom. When this happens, the receiving component is informed and it then invokes the Rte_Rips_Read via the Rte_Read API to perform data inverse transformation via E2EXf/SomelpXf. Reception of byte arrays without Data Transformation is also supported.

Rte_Rips_SRCom relies on Os functionalities with regard to ActivateTask to inform the SW-C that new data has arrived.

Based on the Software Component Descriptions, the RTE provides the Rte_Write and Rte_Read APIs. These are then mapped/redirected to the corresponding Rte_Rips_Write and Rte_Rips_Read APIs from the plugin to access the Rte_Rips_GlobalCopy_myData buffer, which is allocated by the plugin and, if applicable, to invoke transformers for data transformation.

The plugin also provides the notification callback routines in order to support the data transmission/reception through the communication services (e.g. LdCom).

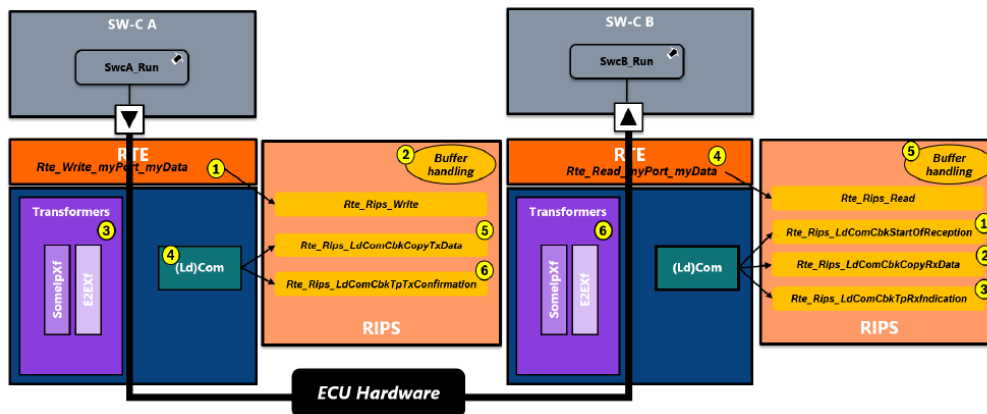


Figure 3.1.Overview

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 Rte_Rips_SRCom

Short Name	Rte_Rips_SRCom
Description	Configuration of the RIPS module to support Inter-ECU S/R Explicit Communication with Transformers via LDCOM_TP.
Multiplicity	0..1
Default Value	-
Type	ECUC-MODULE-DEF

3.3.1.2 Rte_Rips_SRCom/RteRipsPluginFillFlushRoutine/RteRipsPlugin-FillFlushRoutineSymbol

Short Name	RteRipsPluginFillFlushRoutineSymbol
Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.1.3 Rte_Rips_SRCom/RteRipsPluginProps/RtePluginCallback-MemMapPolicy

Short Name	RtePluginCallbackMemMapPolicy
------------	-------------------------------

Description	Enable Generation of Callbacks based on ComSignals/LdComIPdus or SW-Cs
Multiplicity	1..1
Default Value	RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED
Enumeration Literals	RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED, RTE_RIPS_SIGNAL_BASED_MEMMAP_DISABLED
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.4 Rte_Rips_SRCom/RteRipsPluginProps/RtePluginSupportsLd-ComIF

Short Name	RtePluginSupportsLdComIF
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_IF.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.5 Rte_Rips_SRCom/RteRipsPluginProps/RtePluginSupportsLd-ComTP

Short Name	RtePluginSupportsLdComTP
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_TP.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.6 Rte_Rips_SRCom/RteRipsPluginProps/RtePluginSupportsLegacy-MAKW

Short Name	RtePluginSupportsLegacyMAKW
Description	Denotes if or if not the plug-in supports the generation of partition name in Memory Allocation Key Word(Depends on RTE version(>=12.1.0)).
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.7 Rte_Rips_SRCom/RteRipsPluginProps/RtePluginSupportsRead-Write

Short Name	RtePluginSupportsReadWrite
------------	----------------------------

Description	Denotes if or if not the plug-in supports the Rte_Rips_Read/Write API for primitive/complex data.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.8 Rte_Rips_SRCom/RteRipsPluginProps/RteRipsXfrmInvUnqueued-DataCbK

Short Name	RteRipsXfrmInvUnqueuedDataCbK
Description	* If the parameter is set to true: the inverse transformers functions shall be called inside LdComCbKTPRxIndication. * If its set to false: the inverse transformers functions shall be called inside Rte_Read (default workflow).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

3.4 Limitations

The following features are not supported in the current implementation:

- Implicit S/R Communication
- Measurement of LdCom_TP related buffers
- Queued Explicit S/R Communication
- Data Transformation via LdCom_IF (this functionality is natively available in RTA-RTE)
- Fan-In/Fan-out
- PortInterfaceMapping/NetworkRepresentationProps

3.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Rte_Rips_SRCom/RteRipsPluginFillFlushRoutine
- Rte_Rips_SRCom/RteRipsPluginFillFlushRoutine/RteRipsModeDisabling-Handling

- Rte_Rips_SRCom/RteRipsPluginFillFlushRoutine/RteRipsOsSchedulePoint
- Rte_Rips_SRCom/RteRipsPluginProps/RtePluginSupportsIReadIWrite
- Rte_Rips_SRCom/RteRipsPluginProps/RteRipsPluginCommunicationScope
- Rte_Rips_SRCom/RteRipsPluginProps/RteRipsPluginCommunicationType-Support
- Rte_Rips_SRCom/RteRipsPluginProps/RteRipsPluginCommunicationType-Support/RteRipsPluginCommunicationType

3.6 Deviations

3.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1.1 Rte_Rips_SRCom/RteRipsInvocationHandler

[RTA-BSW] Parent Container	Rte_Rips_SRCom/RteRipsInvocationHandler
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocation-Handler
[ADDED] Description	This container defines the configuration container content of an invocation handler of an RTE Implementation Plug-In.
[ADDED] Multiplicity	0..*

3.6.1.2 Rte_Rips_SRCom/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc

[RTA-BSW] Parent Container	Rte_Rips_SRCom/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[RTA-BSW] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables.
[AUTOSAR] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables and triggered runnables via a transformer.

3.6.1.3 Rte_Rips_SRCom/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol

[RTA-BSW] Parent Container	Rte_Rips_SRCom/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
----------------------------	---

[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFnc-Symbol
[RTA-BSW] Description	C-Symbol of the Rte_Rips_InvocationHandler function.
[AUTOSAR] Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.

3.6.1.4 Rte_Rips_SRCom/RteRipsPluginProps

[RTA-BSW] Parent Container	Rte_Rips_SRCom/RteRipsPluginProps
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps
[REMOVED] Introduction	The shortName of the container defines the name of the Rte Implementation Plug-in used for the API infixes.

3.6.1.5 Rte_Rips_SRCom/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy

[RTA-BSW] Parent Container	Rte_Rips_SRCom/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[RTA-BSW] Description	Globally enables or disables the RTE support global copy(s) for each Communication Graph.
[AUTOSAR] Description	Globally enables or disables the support for Rte Implementation Plug-Ins (RIPS)
[REMOVED] Default Value	RTE_RIPS_INSTANTIATION_BY_RTE

3.7 API Definition

3.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

3.8.1 Example Configuration

Rte_Rips_SRCom has only a few configuration parameters. In the example below, it has been configured to work with the TP API from LdCom and a ReadWrite interface from RTE.

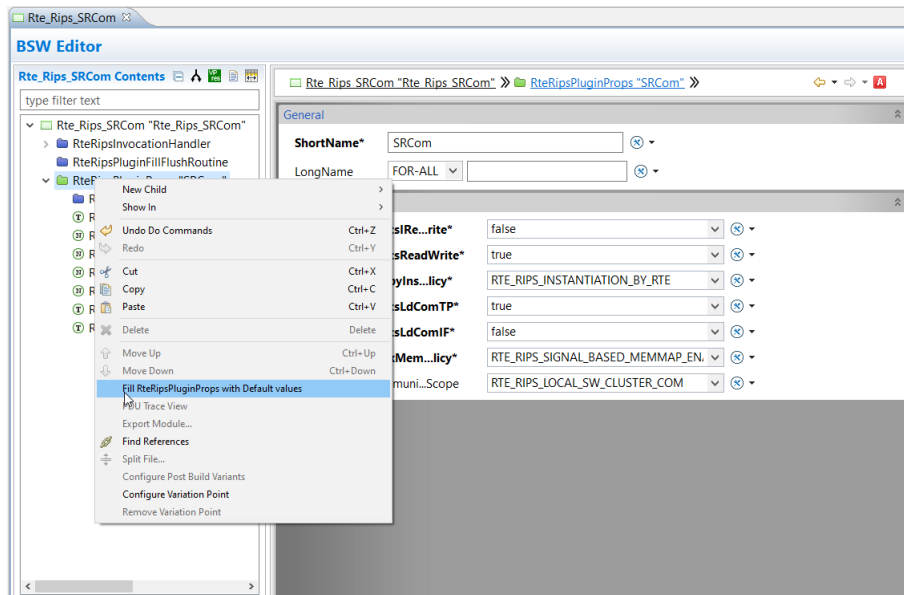


Figure 3.2. Configuration example

3.9 Integration Advice

3.9.1 Init Functions

Rte_Rips_SRCom is initialised by Rte in Rte_Start.

3.9.2 Delnit Functions

No shutdown is required for Rte_Rips_SRCom, but it is de-initialized by Rte in Rte_Stop.

3.9.3 Scheduled Functions

There are no scheduled functions.

3.9.4 Port Mapping

CodeGen for Rte_Rips_SRCom expects the Port mapping in FlatMap/FlatInstanceDescriptor/AnyInstanceRef/ContextElements to be an outerPort (i.e. Port defined in the FlatView) whereas RteGen expects the Port mapping to be an innerPort (i.e. defined in Application SWC).

EcuExtract will generate FlatMap/FlatInstanceDescriptor/AnyInstanceRef/ContextElements with the mapping to innerPort, so you will need to change this to outerPort before running CodeGen. Then, after the CodeGen has been run, and

before running RteGen, change it back to innerPort.

3.9.5 Integration Use Case

As described in the Introduction section for the Rte_Rips module, RIPS is used to create plugin features for RTE in order to make RTE software more modular. For this reason, even though it is considered as a module from BSW and generated by CodeGen, some additional configuration is required for the following:

- Application Software Component
- Create Composition
- System Mapping
- ECU Extract
- AUTOSAR Release
- LdCom configuration
- rba_SysElem configuration
- RTE

3.9.5.1 Application Software Component

The usual SWCs will be required for Inter S/R Communication. For Rte_Rips_SR-Com, re-mapping into a Task is not required.

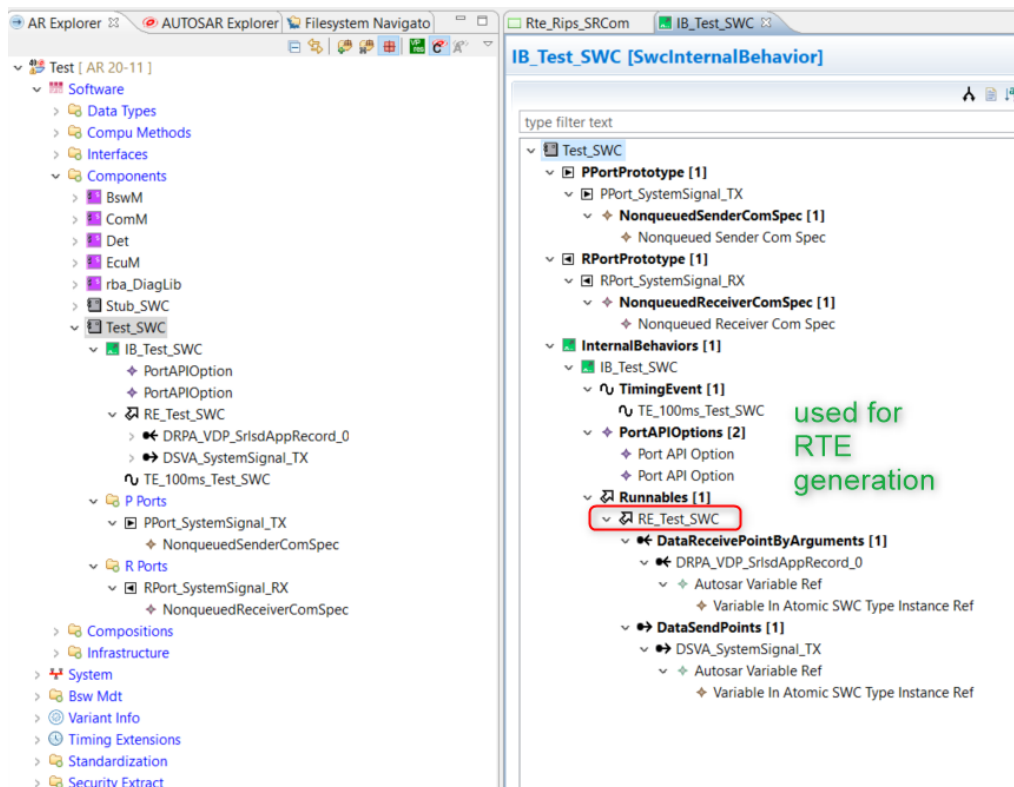


Figure 3.3. Application SWC

3.9.5.2 Create Composition

Create a Component Prototype for the SWC belonging to the Target Ecu.

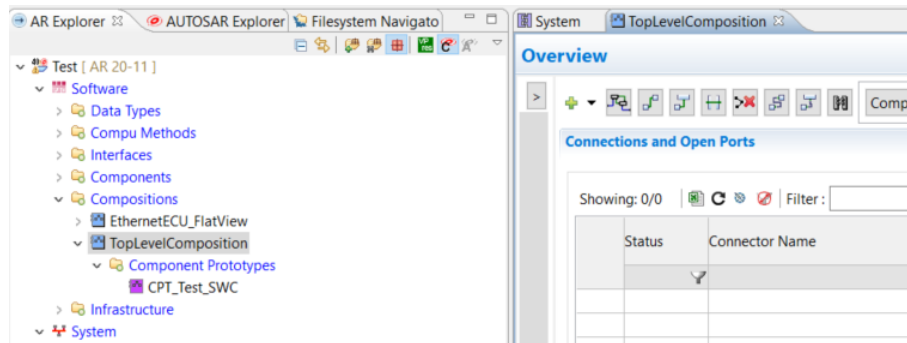


Figure 3.4.Composition

3.9.5.3 System Mapping

Follow the normal workflow for the SWC Prototype that will run on the Target Ecu.

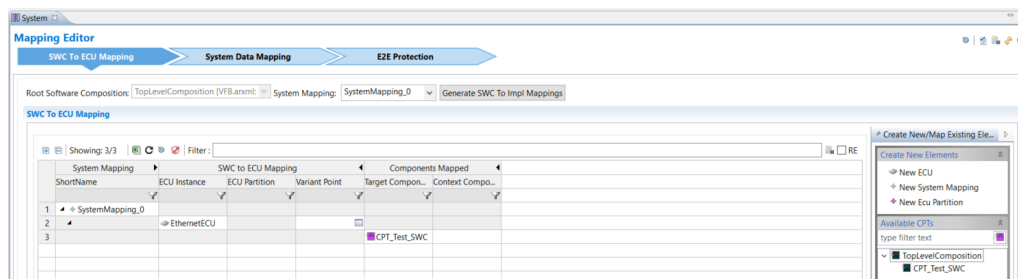


Figure 3.5.System Mapping

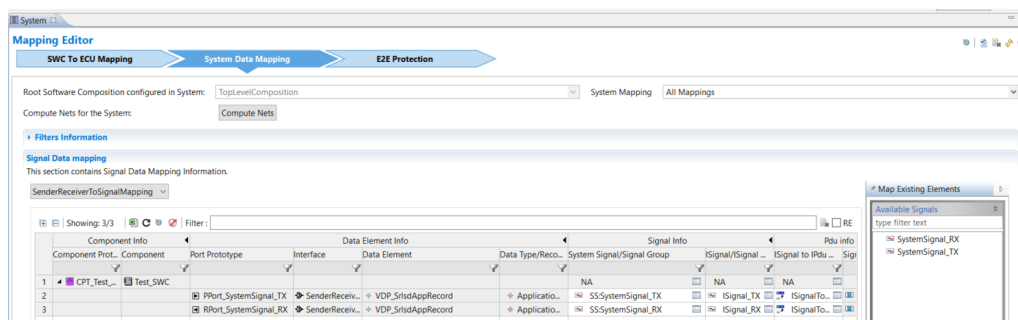


Figure 3.6.System Mapping

3.9.5.4 ECU Extraction

RTE will use FlatInstanceDescriptor to know which Signals are required to

generate the RTE API, so here we are configuring Rips to indicate that these mapping S/R signals will be handled by Rips_SRCom.

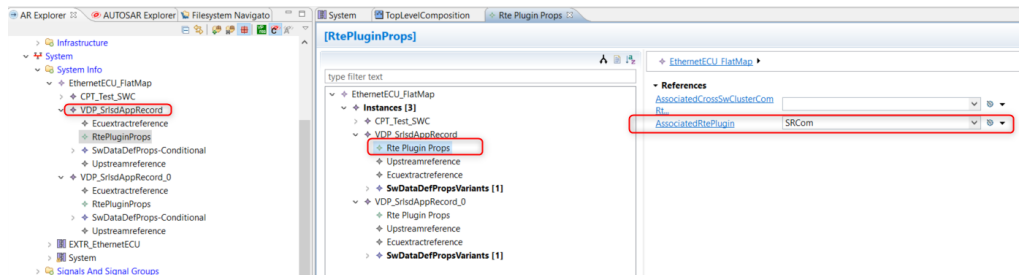


Figure 3.7. RTE Configuration

3.9.5.5 AUTOSAR Release

RtePluginProps requires the project to be configured with latest AR Release.

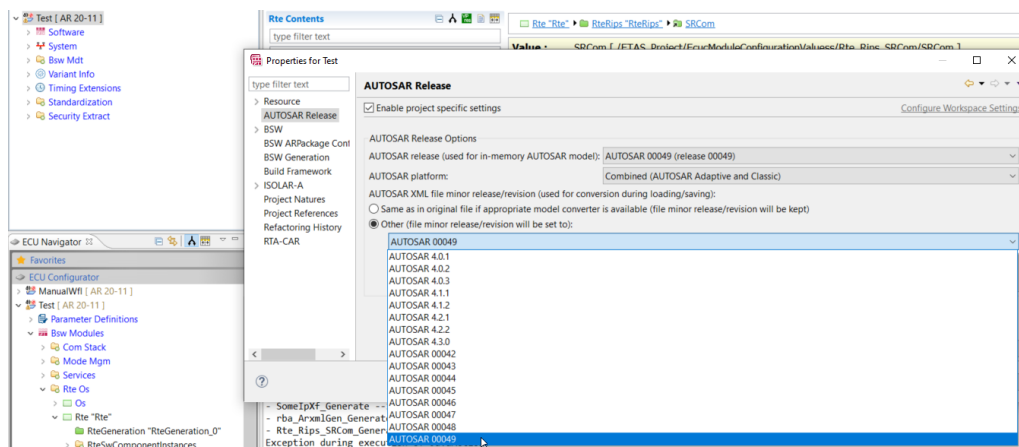


Figure 3.8. Selecting the latest AR Release

3.9.5.6 LdCom

All callback functions from LdCom are configured with the Rte prefix. Following API calls, Rte_Rips_SRCom will create wrappers to handle it with the Rips API.

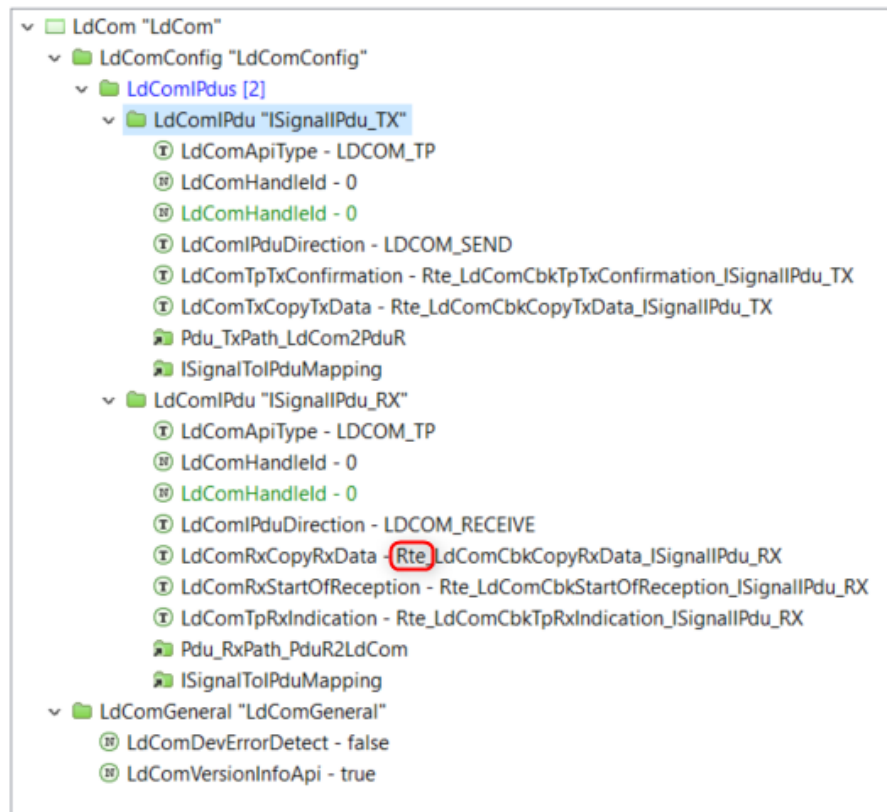


Figure 3.9.LdCom Configuration

3.9.5.7 rba_SysElem (and Xfrm for SomelpXf)

Rte_Rips_SRCom accesses these modules for API references (through Signals from FlastInstanceDescriptors) and in particular the size of buffer shall be handled by Rips though rba_SysElem.



Figure 3.10.rba_SysElem Configuration

3.9.5.8 RTE configuration

Set RIPS ON in the RTE configuration.

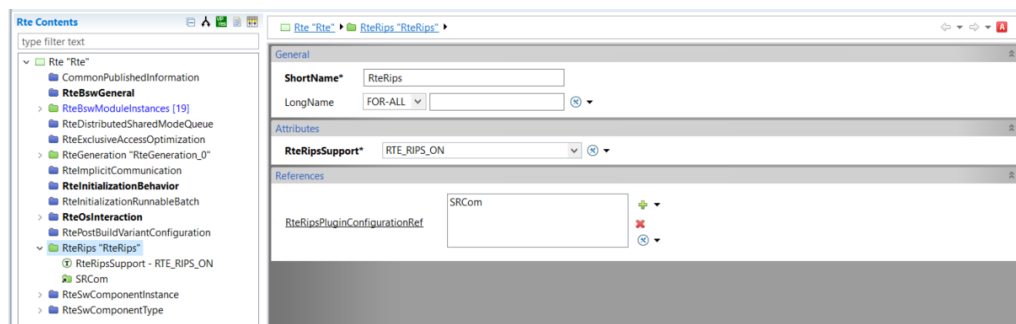


Figure 3.11.Set RIPS ON

You then need to manually update the INDEX for reference SRCom (ISOLAR-A/B currently only links to SRCom, it does not fulfil the configuration with INDEX).

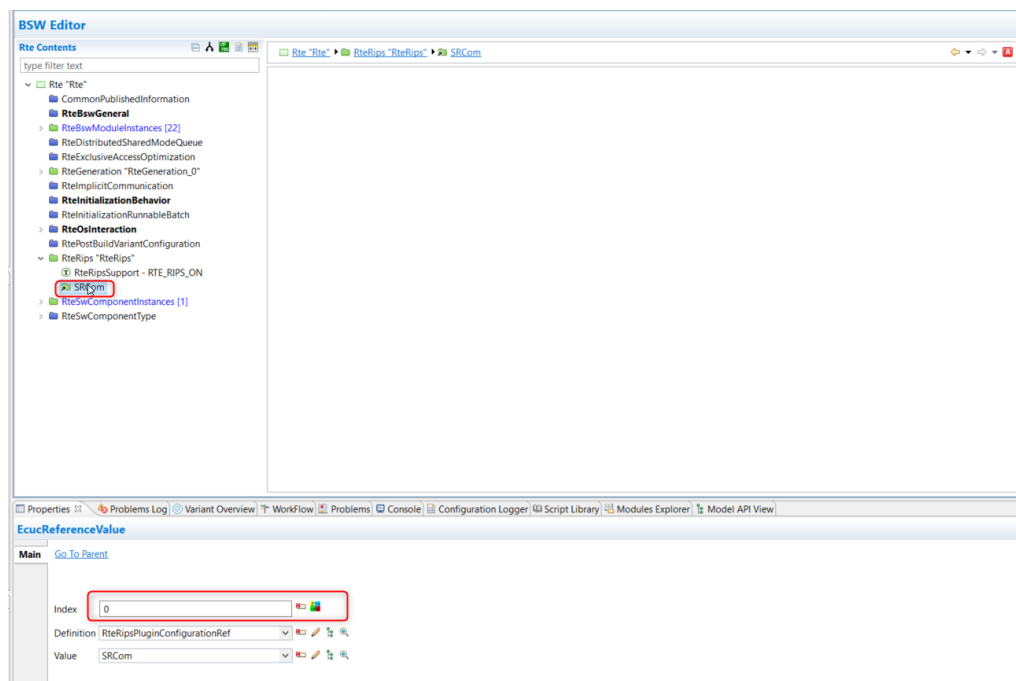


Figure 3.12.Manually update the INDEX

3.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rte_Rips_SRCom module does not use any exclusive OS resources.

3.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

4 RTE Implementation Plugin Support for Signal to Service Translation

Module name	Rte_Rips_Sig2Srv
Package	RTA-RIPSPLUGINS
Module ID	N/A
AUTOSAR Version	22-11

4.1 Introduction

The *RIPS (RTE Implementation Plug-In Service)* for *Signal-to-Service Translation with Transformers* maps signals received from signal-based networks (e.g. the *CAN Bus*) to services transmitted on a service-based network (e.g. the *Ethernet Bus*) and vice versa.

Moreover, the *Rte_Rips_Sig2Srv* module supports the transmission of composite data from ports of various components over the network, as well as the reception of partial/complete data from the network.

A *Require Port (R-Port)* of a component typed by an AUTOSAR Sender/Receiver interface can read the latter's data elements. A *Provide Port (P-Port)* of a component typed by an AUTOSAR Sender/Receiver interface can write the latter's data elements. *Port Interface Mapping*, meanwhile, handles the incompatible interfaces between the *outer port* and Software Component-specific ports (*inner ports*).

When Inter-ECU applies for Sender/Receiver communication, there are many requirements for which the *RIPS* can provide support:

- Transmission/Reception of protected data (*E2EXf*) with *SomeIPXf* and *LdCom(LdCom_IF)/Com*.
- Transmission/Reception of unprotected data with *SomeIPXf* and *LdCom(LdCom_IF)/Com*.
- Transmission/Reception of protected data (*E2EXf*) with *ComXf* and *LdCom(LdCom_IF)/Com*.
- Transmission/Reception of unprotected data with *ComXf* and *LdCom(LdCom_IF)/Com*.
- Data Conversion of *Linear* type for transforming *ISignalgroups* without Data Transformation.

4.1.1 Data Transformation

Transformers give a data transformation mechanism to linearize and transform data. They can be concatenated to *transformer chains* and are executed by the RTE for inter-ECU communication, which is configured to be transformed.

The input of the first transformer in the chain gets the data from the RTE; thus the RTE allocates the buffer and calculates its size using the attribute *BufferComputation*, which describes the "worst case" outcome. Each successive

transformer then uses the output of the preceding one as input. With this approach, functionality like *E2E protection* can be added to the serialized stream.

Currently, AUTOSAR standardizes 3 transformers:

SomelpXf (SOME/IP Transformer)

The SOME/IP transformer is of class *SERIALIZER*, which is invoked by the RTE. It serializes structured data into a linear form. Therefore, it can only be used as the first transformer on the sending side and the last on the receiving side (in execution order).

E2EXfm (E2E Transformer)

The E2E transformer is of class *SAFETY*, invoked by the RTE and responsible for protecting safety-related data elements. On the sender side, the E2E transformer protects the data. On the receiver side, the E2E transformer checks the data and gives the result (via the RTE) to SWCs.

ComBasedXfm (COM-based transformer)

The COM-based transformer serializes structured data into a linear form and adds additional data (e.g. *crc*, *sequence counter*) to it. It can only be used as the first transformer on the sending side and the last transformer on the receiving side.

4.1.2 Interaction

Rte_Rips_Sig2Srv interacts with Transformers (SomelpXf/ComXf and E2EXf) and Com/LdCom (LDCOM_IF) for data (de)serialization and data reception/transmission.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

4.2 Supported Features

Based on the Software Component Descriptions, the RTE provides the *Rte_Write* and *Rte_Read* APIs. These are mapped/redirected to the corresponding *Rte_Rips_Write* and *Rte_Rips_Read* APIs from the plug-in. These, in turn, access the buffer *Rte_Rips_GlobalCopy_myData*, which is allocated by the plug-in and, if applicable, invoke transformers for data transformation.

The plug-in also provides notification callback routines in order to support the data reception through the communication services (e.g. *LdCom*) and the *Transmit* API, which transmits the data collected from various ports via *Com/LdCom*.

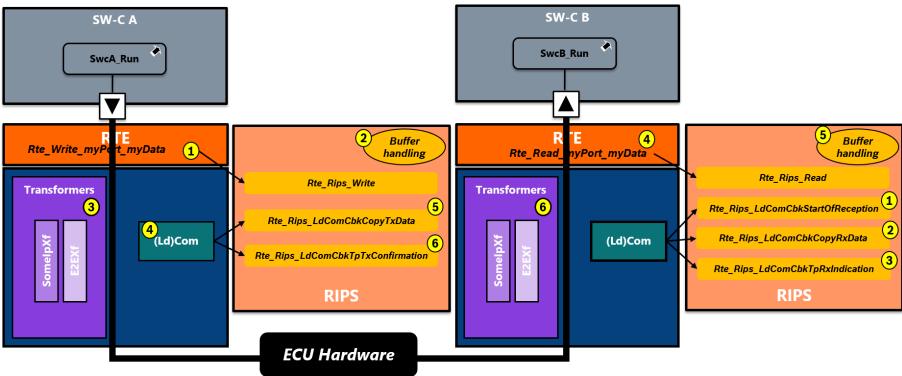


Figure 4.1.Explicit inter-ECU S/R Communication Implementation principle via RIPS

4.2.1 Behavior in Normal Mode

For transmission, a SWC will call Rte_Rips_Sig2Srv_Write via the Rte_Write API to perform data transformation (SomelpXf/ComXf and E2EXf) and send the transformed data to the receiving SWC via LdCom(IF)/Com.

For reception, a transformed data is received by the Rte_Rips_Sig2Srv from LdCom. The receiving SWC is informed and it then invokes Rte_Rips_Sig2Srv_Read via the Rte_Read API to perform data inverse transformation (SomelpXf/ComXf and E2EXf).

4.3 Extensions

4.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.1.1 Rte_Rips_Sig2Srv

Short Name	Rte_Rips_Sig2Srv
Description	Configuration of the RIPS module to support Inter-ECU S/R Explicit Communication with Transformers via LDCOM_TP.
Multiplicity	0..1
Default Value	-
Type	ECUC-MODULE-DEF

4.3.1.2 Rte_Rips_Sig2Srv/RteRipsPluginFillFlushRoutine/RteRipsPluginFillFlushRoutineSymbol

Short Name	RteRipsPluginFillFlushRoutineSymbol
Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.
Multiplicity	1..1

Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

4.3.1.3 Rte_Rips_Sig2Srv/RteRipsPluginProps/RtePluginCallback-MemMapPolicy

Short Name	RtePluginCallbackMemMapPolicy
Description	Enable Generation of Callbacks based on ComSignals/LdComIPdus or SW-Cs
Multiplicity	1..1
Default Value	RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED
Enumeration Literals	RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED, RTE_RIPS_SIGNAL_BASED_MEMMAP_DISABLED
Type	ECUC-ENUMERATION-PARAM-DEF

4.3.1.4 Rte_Rips_Sig2Srv/RteRipsPluginProps/RtePluginSupportsLd-ComIF

Short Name	RtePluginSupportsLdComIF
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_IF.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.5 Rte_Rips_Sig2Srv/RteRipsPluginProps/RtePluginSupportsLd-ComTP

Short Name	RtePluginSupportsLdComTP
Description	Denotes if or if not the plug-in supports the (a)synchronous C/S Data Transformation via LDCOM_TP.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.6 Rte_Rips_Sig2Srv/RteRipsPluginProps/RtePluginSupportsLegacy-MAKW

Short Name	RtePluginSupportsLegacyMAKW
Description	Denotes if or if not the plug-in supports the generation of partition name in Memory Allocation Key Word(Depends on RTE version(>=12.1.0)).
Multiplicity	1..1

Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.7 Rte_Rips_Sig2Srv/RteRipsPluginProps/RtePluginSupportsRead-Write

Short Name	RtePluginSupportsReadWrite
Description	Denotes if or if not the plug-in supports the Rte_Rips_Read/Write API for primitive/complex data.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.8 Rte_Rips_Sig2Srv/RteRipsPluginProps/RteRipsSig2SrvBuffersUseDataElementName

Short Name	RteRipsSig2SrvBuffersUseDataElementName
Description	Enables Generation of GlobalCopies based on DataPrototype ShortName
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.4 Limitations

Version v12.7.0 has following limitations:

- Variable Data Prototypes handling Data Conversion can only reference Application Datatypes. Only CompuMethods of category *Linear* are supported.

4.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Rte_Rips_Sig2Srv/RteRipsPluginFillFlushRoutine
- Rte_Rips_Sig2Srv/RteRipsPluginFillFlushRoutine/RteRipsModeDisabling-Handling
- Rte_Rips_Sig2Srv/RteRipsPluginFillFlushRoutine/RteRipsOsSchedulePoint
- Rte_Rips_Sig2Srv/RteRipsPluginProps/RtePluginSupportsIReadIWrite
- Rte_Rips_Sig2Srv/RteRipsPluginProps/RteRipsPluginCommunicationScope
- Rte_Rips_Sig2Srv/RteRipsPluginProps/RteRipsPluginCommunicationType-Support
- Rte_Rips_Sig2Srv/RteRipsPluginProps/RteRipsPluginCommunicationType-

Support/RteRipsPluginCommunicationType

4.6 Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

4.6.1 Rte_Rips_Sig2Srv/RteRipsInvocationHandler

[RTA-BSW] Parent Container	Rte_Rips_Sig2Srv/RteRipsInvocationHandler
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler
[ADDED] Description	This container defines the configuration container content of an invocation handler of an RTE Implementation Plug-In.
[ADDED] Multiplicity	0..*

4.6.2 Rte_Rips_Sig2Srv/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc

[RTA-BSW] Parent Container	Rte_Rips_Sig2Srv/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[RTA-BSW] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables.
[AUTOSAR] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables and triggered runnables via a transformer.

4.6.3 Rte_Rips_Sig2Srv/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol

[RTA-BSW] Parent Container	Rte_Rips_Sig2Srv/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
[RTA-BSW] Description	C-Symbol of the Rte_Rips_InvocationHandler function.
[AUTOSAR] Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.

4.6.4 Rte_Rips_Sig2Srv/RteRipsPluginProps

[RTA-BSW] Parent Container	Rte_Rips_Sig2Srv/RteRipsPluginProps
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps
[REMOVED] Introduction	The shortName of the container defines the name of the Rte Implementation Plug-in used for the API infixes.

4.6.5 Rte_Rips_Sig2Srv/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy

[RTA-BSW] Parent Container	Rte_Rips_Sig2Srv/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[RTA-BSW] Description	Globally enables or disables the RTE support global copy(s) for each Communication Graph.
[AUTOSAR] Description	Globally enables or disables the support for Rte Implementation Plug-Ins (RIPS)
[REMOVED] Default Value	RTE_RIPS_INSTANTIATION_BY_RTE

4.7 API Definition

List of supported API functions:

Table 4.15.Rte_Rips_Sig2Srv Supported API functions

Rte_Rips_Sig2Srv_Write	Called by SWC via Rte_Write to perform data transformation and send it to the receiving SWC.
Rte_Rips_Sig2Srv_Read	Upon data reception, called by SWC via Rte_Read to perform data inverse transformation.
Rte_Rips_Sig2Srv_Transmit	Transmits the data collected from various ports via Com/LdCom.
Rte_Rips_Sig2Srv_NotifyRxAck	Notification callback function for the reception of data from the bus via COM or LdCOM-IF.
Rte_Rips_Sig2Srv_DRead	
Rte_Rips_Sig2Srv_LdComCbKtpTxConfirmation	Callback function confirming the transmission of data from the bus via LdCOM-TP.
Rte_Rips_Sig2Srv_LdComCbKCopyTxData	Callback function for the copying of transmission data from the bus via LdCOM-TP.
Rte_Rips_Sig2Srv_LdComCbKtpRxIndication	Callback function for the reception of data from the bus via LdCOM-TP.

Rte_Rips_Sig2Srv_LdComCbkJCopyRxData	Callback function for the copying of reception of data from the bus via LdCOM-TP.
Rte_Rips_Sig2Srv_LdComCbkJStartOfReception	Callback function for the start of reception of data from the bus via LdCOM-TP.

4.8 Configuration Advice

4.8.1 Memory Mapping

Rte_Rips_Sig2Srv's implementation includes a local module memory mapping header file, *Rte_Rips_Sig2Srv_MemMap.h*, which includes the software addressing methods of CUCEL from the file *rba_CUCELSwAddrMethod_BSWMD.arxml*. This makes it possible to map Rte_Rips_Sig2Srv code/data to specific memory sections.

Rte_Rips_Sig2Srv uses the following memory allocation keywords when RtePluginCallbackMemMapPolicy is set to *RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED*:

Rte_Rips_Sig2Srv Memory Mapping Sections

SI No.	Start/End of Section	Memory Allocation Keywords	
1	Start of section	RTE_RIPS_SIG2SRV_MAIN_START_SEC_CODE	Store/map Rte_Rips_Sig2Srv_Start and Rte_Rips_Sig2Srv_Stop lifecycle APIs.
	End of section	RTE_RIPS_SIG2SRV_MAIN_STOP_SEC_CODE	
2	Start of section	RTE_RIPS_SIG2SRV_SIG_<LDCOMIPDU>_START_SEC_CODE	Store/map Rte_Rips_Sig2Srv callback functions for transmission/reception.
	End of section	RTE_RIPS_SIG2SRV_SIG_<LDCOMIPDU>_STOP_SEC_CODE	
3	Start of section	RTE_RIPS_SIG2SRV_<SWC>_START_SEC_CODE	Store/map Rte_Rips_Sig2Srv_Write and Rte_Rips_Sig2Srv_Read APIs.
	End of section	RTE_RIPS_SIG2SRV_<SWC>_STOP_SEC_CODE	
4	Start of section	RTE_RIPS_SIG2SRV_<SWC>_START_SEC_VAR_INIT_8	Store/map variables used to handle transmission/reception status that are initialized after every reset.
	End of section	RTE_RIPS_SIG2SRV_<SWC>_STOP_SEC_VAR_INIT_8	
5	Start of section	RTE_RIPS_SIG2SRV_<SWC>_START_SEC_VAR_INIT_16	Store/map variables used to keep the buffer-Length of the data used to perform serialization initialized with values after every reset.
	End of section	RTE_RIPS_SIG2SRV_<SWC>_STOP_SEC_VAR_INIT_16	
6	Start of section	RTE_RIPS_SIG2SRV_<SWC>_START_SEC_VAR_CLEARED_8	Store/map byte array variables used to perform data (de)serialization initialized with 0 after every reset.
	End of section	RTE_RIPS_SIG2SRV_<SWC>_STOP_SEC_VAR_CLEARED_8	
7	Start of section	RTE_RIPS_SIG2SRV_<SWC>_START_SEC_VAR_CLEARED_16	Store/map variables used to keep the buffer-Length of the data used to perform (de)serialization initialized with 0 after every reset.

	End of section	RTE_RIPS_SIG2SRV_<SWC>_STOP_SEC_VAR_CLEARED_16	
8	Start of section	RTE_RIPS_SIG2SRV_CBK_<SWC>_START_SEC_CALLOUT_CODE	Store/map Rte_Rips_Sig2Srv_NotifyRxAck_<XXXX>([...]) callbacks when RtePluginCallbackMemMapPolicy set to RTE_RIPS_SIGNAL_BASED_MEMMAP_DISABLED.
	End of section	RTE_RIPS_SIG2SRV_CBK_<SWC>_STOP_SEC_CALLOUT_CODE	

4.8.2 Dependencies

Package/Component	Purpose
Rte_Rips	Rte_Rips_Sig2Srv uses the Rte_Rips_MemCpy function to copy complex data types.
CUCEL	Rte_Rips_Sig2Srv relies on the availability of the SwAddrMethods, ComStack_Types.h etc.
Transformers	Rte_Rips_Sig2Srv reads the SomelpXf module configuration to call the respective SomelpXf APIs for Data Transformation. Rte_Rips_Sig2Srv reads the ComXf module configuration to call the respective ComXf APIs for Data Transformation.
E2EServices	Rte_Rips_Sig2Srv reads the E2EXf module configuration to call the respective E2EXf APIs for Data Transformation.
ComServices	Rte_Rips_Sig2Srv reads the (Ld)Com module configuration which contains the Signal name used to call the respective (Ld)Com APIs.
EbOs	Rte_Rips_Sig2Srv relies on Os functionalities with respect to ActivateTask to inform the SWC that new data has arrived.

4.8.3 Additional Hints

The Project property "com.bosch.bsw.merge_artop_elements" should be set to TRUE in the *ide_properties.txt* file to access the ARTOP model from multiple input .ARXML files.



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

4.9 Integration Advice

4.9.1 How To Integrate

RTE

When a *Runnable Entity* is triggered by a *DataReceivedEvent* and reads *LdCom*-relevant data, the integrator will ensure that the respective Event is mapped onto an *OsTask* where *TimingEvents* are also mapped. This is to ensure that the *Runnable Entity* will be activated accordingly by the RTE based on flags.

EcuC

The *EcuCRbArRelease* parameter is to be set accordingly (*AR42_Rev2/AR45_Rev0*).

Rte_Rips_Sig2Srv

The *RtePluginCallbackMemMapPolicy* parameter is to be set accordingly (*RTE_RIPS_SIGNAL_BASED_MEMMAP_ENABLED/RTE_RIPS_SIGNAL_BASED_MEMMAP_DISABLED*)

4.9.2 Init Functions

The initialization of *Rte_Rips_Sig2Srv* is done by the RTE module within *Rte_Start()*.

4.9.3 Delnit Functions

No shutdown is required for *Rte_Rips_Sig2Srv*. However its de-initialization is done by the RTE module within *Rte_Stop()*.

4.9.4 Scheduled Functions

There are no scheduled functions.

4.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The *Rte_Rips_Sig2Srv* module does not use any exclusive OS resources.

4.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

5 RTE Implementation Plugin Support for Safety C/S Communication

Module name	Rte_Rips_CSSafety
Package	RTA-RIPSPLUGINS
AUTOSAR Module ID	n/a
AUTOSAR Version	22-11

5.1 Introduction

The Rte_Rips_CSSafety Component is based on the RIPS approach to enable freedom from interference for C/S communication between OsApplications - with different ASIL levels - on the same resp. different OsCore(s).

The AUTOSAR supports Client-Server communication whereby a client invokes a function (the server) in another SWC instance. Depending on the configuration and location of client and server the RTE implementation may use a direct or trusted function call, RTE managed queues or IOC to communicate the request from the client to server and the reply from server to client.

The AUTOSAR SWS RTE permits optimization of client-server communication to a direct (See [SWS_Rte_07409]) or trusted (See [SWS_Rte_08904]) function invocation when all of the following conditions are true:

- The server runnable is marked as CanBeInvokedConcurrently.
- Client and server are in the same partition (i.e. call is intra-partition) or in different partitions (i.e. call is inter-partition).
- The C/S call is synchronous.
- The OperationInvokedEvent is not mapped to a task.

RTA-RTE currently supports [SWS_Rte_07409] which optionally optimizes inter-partition calls to direct function invocation, which is in contradiction of [SWS_Rte_08904] which states that it shall use Os Service CallTrustedFunction. However [SWS_Rte_08904] contradicts the OS specification which does not support the usage of CallTrustedFunction for the use case where Client and Server are located in different OsApplications on different Cores.

- Trusted to Trusted

When Client and Server have both unrestricted access to memory direct invocation of the server operation by the client is feasible. No special handling is therefore required.

- Trusted to Untrusted

A trusted OsApplication has unrestricted access to memory and therefore optimisation of the client-server communication to a direct function call of an untrusted server is technically possible even if it is violation of [SWS_Rte_08904]. However such an optimization is unsafe since the "trusted" attribute should not be applied to the "untrusted" server code by executing it in the context of the client.

Therefore the RTE should use the `CallTrustedFunction` to ensure that the server is executed in the correct context and thus untrusted server does inherit the trusted execution properties of the client.

- Untrusted to Trusted

A client in an untrusted `OsApplication` cannot directly invoke the server in another `OsApplication`. Therefore the same approach as for Trusted to Untrusted has to be followed.

- Untrusted to Untrusted

This use case is similar to the trusted to untrusted one and therefore usage of `OS CallTrustedFunction` AP applies.

Whenever there is a data transfer between Client and Server across `OsApplication/EcucPartition` boundaries the RTE will make use of marshalling and IOC channels:

- **The RTE API will marshal the IN and INOUT parameters provided by the Client before transferring control to the server.**
- **The trusted function will receive the IN and INOUT parameters from the Client, un-marshall them before passing them to the Server.**
- **The trusted function will transmit the INOUT/OUT parameters and, if applicable, the return value from the Server to the Client using IOC (e.g. `locWrite`).**
- **The Client API will retrieve the INOUT/OUT parameters and, if applicable, the return value from the Server using IOC (e.g. `locRead`).**

The `CallTrustedFunction` API requires that IN/INOUT parameters are marshalled by the client's RTE API call before transferring control to the server and the INOUT/OUT parameters and return value are provided via IOC on reply. This approach avoids that the untrusted server writes in the the data belonging to the trusted `OsApplication`.

`RIPS_CSSafety` also satisfies requirement when Server operation is not marked as `canbeinvokedconcurrently`. In this scenario client invoke API will activate task mapped to operation invoked event mapped to server operation and data transfer is performed via IOC.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

5.2 Supported Features

- Intra-ECU C/S Communication between `OsApplications` on the same `OsCore` via `CallTrustedFunction` resp. IOC.
- Intra-ECU C/S Communication between `OsApplications` on different `OsCores` via `ReceiverPullCB` resp. IOC.
- Timeout Handling.

- Synchronous Client server communication when Server operation is nonreentrant.

5.3 Extensions

5.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

5.3.1.1 Rte_Rips_CSSafety

Short Name	Rte_Rips_CSSafety
Description	Configuration of the RIPS module to support Intra-ECU C/S Communication over OsApplication boundaries.
Multiplicity	0..1
Default Value	-
Type	ECUC-MODULE-DEF

5.3.1.2 Rte_Rips_CSSafety/RteRipsPluginFillFlushRoutine/RteRipsPluginFillFlushRoutineSymbol

Short Name	RteRipsPluginFillFlushRoutineSymbol
Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.3 Rte_Rips_CSSafety/RteRipsPluginProps/RtePluginSupportsInterCoreWithoutTimeout

Short Name	RtePluginSupportsInterCoreWithoutTimeout
Description	Denotes if or if not the plug-in shall support Inter-Core C/S Communication without Timeout.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.4 Rte_Rips_CSSafety/RteRipsPluginProps/RtePluginSupportsOptimizedIOC

Short Name	RtePluginSupportsOptimizedIOC
------------	-------------------------------

Description	Denotes if or if not the plug-in supports optimized or standard IOC communication. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.5 Rte_Rips_CSSafety/RteRipsPluginProps/RtePluginSupportsTimeout

Short Name	RtePluginSupportsTimeout
Description	Denotes if or if not the plug-in supports timeout handling over core boundaries.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.6 Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsTimeOutHandlingProps

Short Name	RteRipsTimeOutHandlingProps
Description	A list of OsTasks (one per OsApplication/EcucPartition) which will be used to invoke the MainFunctions generated by the RTE to perform timeout w.r.t CSSafety communication.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.7 Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsTimeOutHandlingProps/RteRipsTimeOutHandlingOsTaskRef

Short Name	RteRipsTimeOutHandlingOsTaskRef
Description	A reference to an OsTask that will be used to handle timeout for all clients allocated to the EcucPartition resp. OsApplication containing the referenced OsTask.
Multiplicity	0..*
Default Value	-
Type	ECUC-REFERENCE-DEF

5.3.1.8 Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsTime-

OutHandlingProps/RteRipsTimeOutImplementation

Short Name	RteRipsTimeOutImplementation
Description	Denotes if or if not the Timeout Handle used instead of OsAlarm functions in the Generation.
Multiplicity	0..1
Default Value	OSALARMCBK_APPROACH
Enumeration Literals	OSALARMCBK_APPROACH, MAINFUNCTION_APPROACH
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

5.4 Limitations

5.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Rte_Rips_CSSafety/RteRipsPluginFillFlushRoutine
- Rte_Rips_CSSafety/RteRipsPluginFillFlushRoutine/RteRipsModeDisabling-Handling
- Rte_Rips_CSSafety/RteRipsPluginFillFlushRoutine/RteRipsOsSchedule-Point
- Rte_Rips_CSSafety/RteRipsPluginProps/RtePluginSupportsIReadIWrite
- Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsPluginCommunication-Scope
- Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsPluginCommunication-TypeSupport
- Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsPluginCommunication-TypeSupport/RteRipsPluginCommunicationType

5.6 Deviations

5.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

5.6.1.1 Rte_Rips_CSSafety/RteRipsInvocationHandler

[RTA-BSW] Parent Container	Rte_Rips_CSSafety/RteRipsInvocationHandler
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler
[ADDED] Multiplicity	0..*

5.6.1.2 Rte_Rips_CSSafety/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc

[RTA-BSW] Parent Container	Rte_Rips_CSSafety/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc
[RTA-BSW] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables.
[AUTOSAR] Description	This container describes an invocation handler function implemented by an RTE Implementation Plug-In to handle the invocation of server runnables and triggered runnables via a transformer.

5.6.1.3 Rte_Rips_CSSafety/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol

[RTA-BSW] Parent Container	Rte_Rips_CSSafety/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsInvocationHandler/RteRipsInvocationHandlerFnc/RteRipsInvocationHandlerFncSymbol
[RTA-BSW] Description	C-Symbol of the Rte_Rips_InvocationHandler function.
[AUTOSAR] Description	C-Symbol of the Rte_Rips_FillFlushRoutine function.

5.6.1.4 Rte_Rips_CSSafety/RteRipsPluginProps

[RTA-BSW] Parent Container	Rte_Rips_CSSafety/RteRipsPluginProps
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps
[REMOVED] Introduction	The shortName of the container defines the name of the RTE Implementation Plug-in used for the API infixes.

5.6.1.5 Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy

[RTA-BSW] Parent Container	Rte_Rips_CSSafety/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[AUTOSAR] Parent Container	EcucDestinationUriDefSets/RteRipsUriDefSet/RteRipsPlugin/RteRipsPluginProps/RteRipsGlobalCopyInstantiationPolicy
[RTA-BSW] Description	Globally enables or disables the support for Rte Implementation Plug-Ins (RIPS) Tags: atp.Status=draft
[AUTOSAR] Description	Globally enables or disables the support for Rte Implementation Plug-Ins (RIPS)

5.7 API Definition

5.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

5.9 Integration Advice

5.9.1 Init Functions

The initialization of Rte_Rips_CSSafety is done by the RTE (Run Time Environment) module within the Rte_Start().

5.9.2 Delnit Functions

No shutdown is required for Rte_Rips_CSSafety. However its de-initialization is done by the RTE (Run Time Environment) module within the Rte_Stop().

5.9.3 Scheduled Functions

When a timeout applies, the core-specific *Rte_Rips_CSSafety_Tick_Timeouts* interrupt service routines shall be called accordingly, at a fixed rate of 1ms in ISR of category 2.

There is no fixed scheduling policy for the usage of the APIs.

Timeout Handling

Prior to v12.4.0, timeout handling was only implemented based upon *OS ALARM Callbacks*. This approach still works, but in projects where there is/will be a huge

number of such callbacks, runtime performance and overall system behavior might be adversely affected. Therefore, an additional mechanism based upon periodic Main Functions has been implemented so that a Project can now select based upon which implementation best suits its needs:

- OS Alarm Callback Approach

This is the default implementation method when the parameter `RteRipsTimeoutHandlingProps / RteRipsTimeoutImplementation` is either not configured, or configured and set to `OSALARMCBK_APPROACH`. When selected, no further activities are needed.

- Main Function Approach

This implementation method is enabled when the parameter `RteRipsTimeoutHandlingProps / RteRipsTimeoutImplementation` is configured and set to `MAINFUNCTION_APPROACH`. When selected:

- For each available *OsApplication/EcuCPartition*, exactly 1 *OsTask* via `RteRipsTimeoutHandlingProps / RteRipsTimeoutHandlingOsTaskRef` needs to be configured. This is the *OsTask* which will schedule the generated *MainFunction* required to perform timeout handling for all SWCs allocated to the corresponding partition.

Configuration of Main Function-based approach:

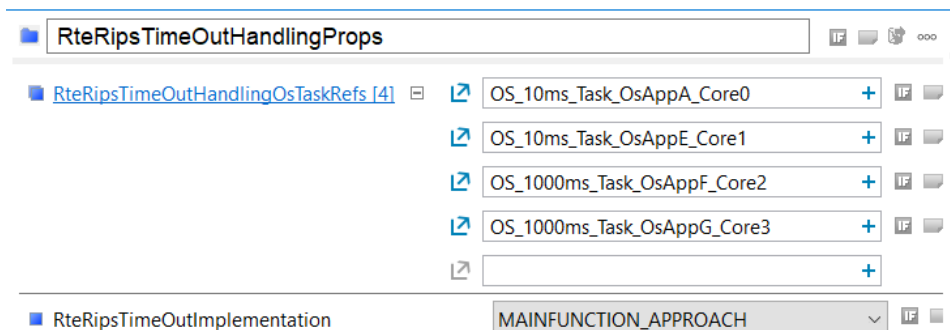


Figure 5.1. RTE Timeout handling using Main Function

- For each referenced *OsTask* in the context of `RteRipsTimeoutHandlingProps / RteRipsTimeoutHandlingOsTaskRef`, the periodicity of that *OsTask* needs to be defined via the attribute `OsTaskPeriod`. The configured value will be used to generate the counter variables, as well as to determine the `TimingEvent.Period` of the required *MainFunction*.

Task periodicity:

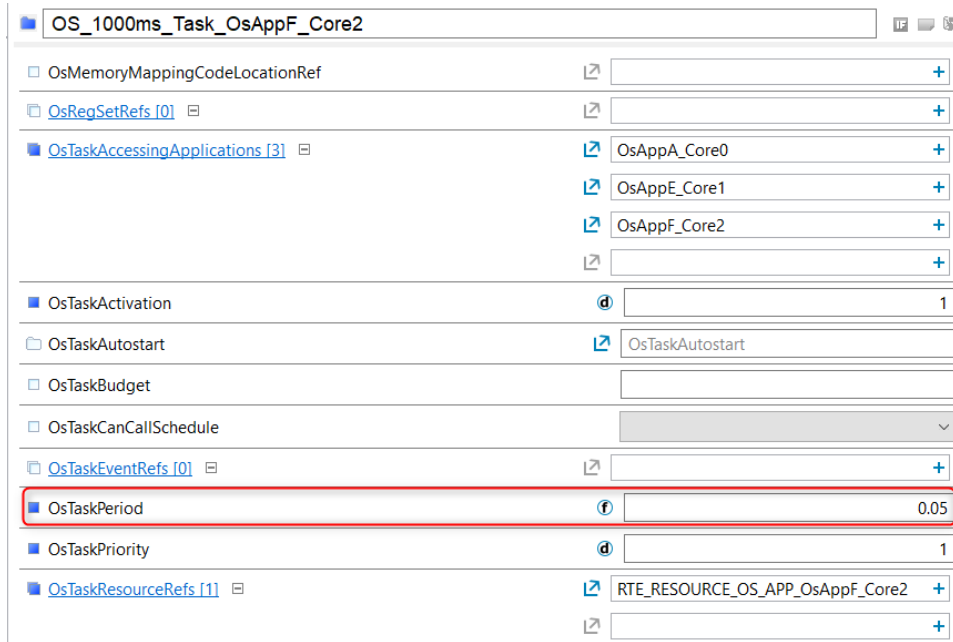


Figure 5.2. OsTaskPeriod sets periodicity

- After BSW Code Generation, the generated *Rte_Rips_CSXfrm_Cfg_BSWMD.arxml* file will contain MainFunctions configured as BswSchedulableEntities being triggered via BswTimingEvents. The *Schedulable Entities* have to be mapped to the respective *OsTasks*:

Mapping MainFunction to OsTask:

OsTask	Entities	Compon...	Event
OS_1000ms_Task_OsAppF_C...	- Rte_Rips_CSSafety_MainFunction_EcucPartitionC_Core2_50ms - RE_rba_SwcClient_James_1000ms	- BSWIM... - rba_Sw...	- TE_Rte_Rips_CSSafety_MainFunci... - TE_rba_SwcClient_James_1000ms
OS_1000ms_Task_OsAppG_C...	- Rte_Rips_CSSafety_MainFunction_EcucPartitionD_Core3_500ms - RE_rba_SwcClient_Jenny_1000ms	- BSWIM... - rba_Sw...	- TE_Rte_Rips_CSSafety_MainFunci... - TE_rba_SwcClient_Jenny_1000ms
OS_10ms_Task_OsAppA_Cor...	- Rte_Rips_CSSafety_MainFunction_EcucPartitionA_Core0_5ms - MainFunction	- BSWIM... - CPT_Nv...	- TE_Rte_Rips_CSSafety_MainFunci... - TE_NvM_MainFunction_10ms
OS_10ms_Task_OsAppE_Core1			

Figure 5.3. Mapping Schedulable Entities to Tasks

5.9.4 Integration

The *Rte_Rips_CSSafety* Component interacts only with OS resp. IOC, and the integrator should have basic knowledge of AUTOSAR and on how to configure the RTE in general.

- OS:

- For systems relying on OS with SC > SC1, Support of *OsAlarmCall-backs* should be explicitly enabled.

5.9.5 Intra Core

As shown in the image, a simplified interaction of the C/S Communication where Client and Server are located on different OsApplications on the same Core. When the Application invokes the respective Rte_Call API the RTE redirects it to Rte_Rips_<Plugin>_Invoke API which is implemented by the RIPS plugin.

This is now responsible for the marshalling/un-marshalling of data and the invocation of the server via the CallTrustedFunction. After the server has executed the request it sends the response back via IOC (locWrite) to the Client which read the data before providing it to the application.

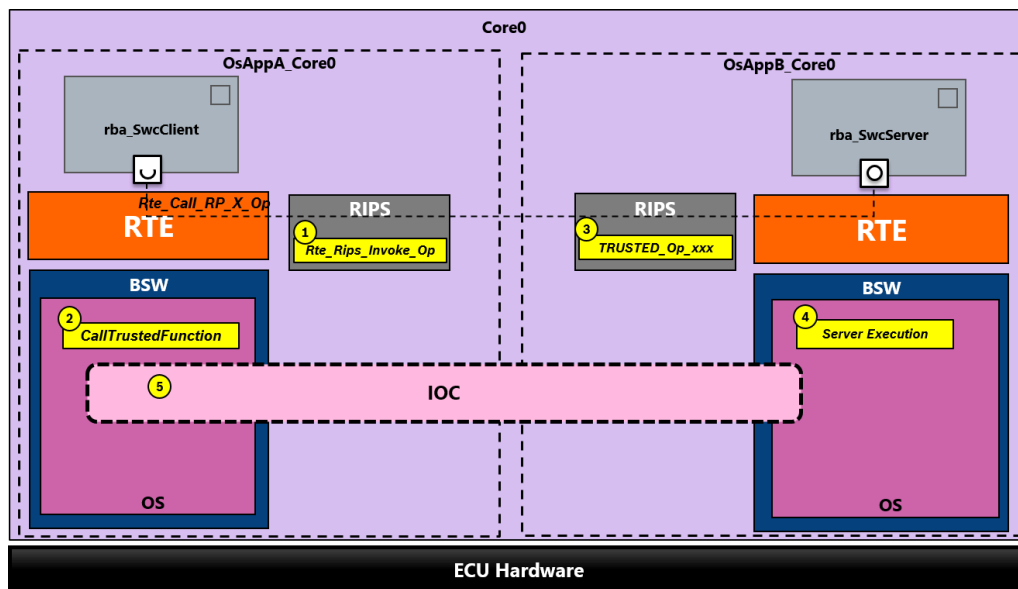


Figure 5.4. Intra Core C/S Implementation Principle via RIPS

5.9.6 Inter Core

When the Client and Server are located to OsApplications on different Cores, a different approach, see image below is followed. There the Client transfers the Input parameters via IOC (instead of CallTrustedFunction), with a notification callback configured running on the receiving core.

That callback runs in a normal implementation of CallTrustedFunction while executing the client request and sends an IOC return message with a different notification callback to the Client to signify completion.

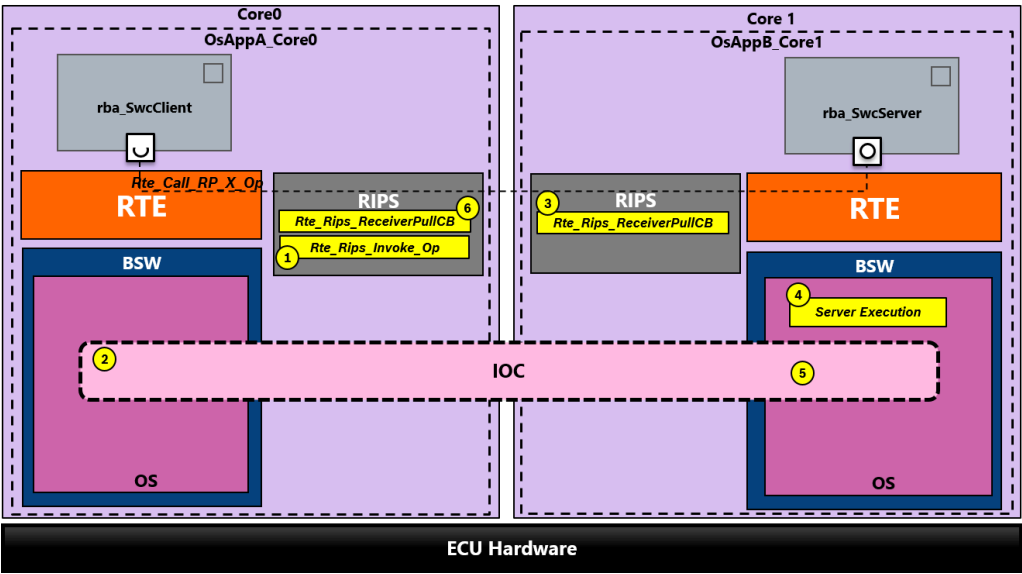


Figure 5.5. Inter Core C/S Implementation Principle via RIPS

5.9.7 Dependencies

Package/Component	Purpose
Rte_Rips	The Rte_Rips_CSSafety uses the Rte_Rips_MemCpy function to copy complex data types and some macros timeout handling.
CUCEL	The Rte_Rips_CSSafety relies on the availability the SwAd-drMethods etc.
EbOs	The Rte_Rips_CSSafety relies on Os functionalities w.r.t CallTrusted-Function, ReceiverPullCB, OsAlarm resp. OsCounter. For data transfer it relies on loc Communication.

5.9.8 Miscellaneous

Note: In case of interpartition client-server communication over core boundaries it is highly recommended to always specify a timeout>0 when synchronous server calls are used. Otherwise the the client on the calling core would wait for the server response infinitely.

5.9.9 Integration Use Case

As described in the Introduction section for the Rte_Rips module, RIPS is used to create plugin features for RTE in order to make RTE software more modular. For this reason, even though it is considered as a module from BSW and generated by CodeGen, some additional configuration is required for the following:

- Application Software Component
- ECU Extraction
- Create EcucPartition

- Create OsApplication
- Mapping SWC into EcucPartition
- RTE

5.9.9.1 Application Software Component

The following SWCs will be required when working with Rte_Rips_CSSafety.

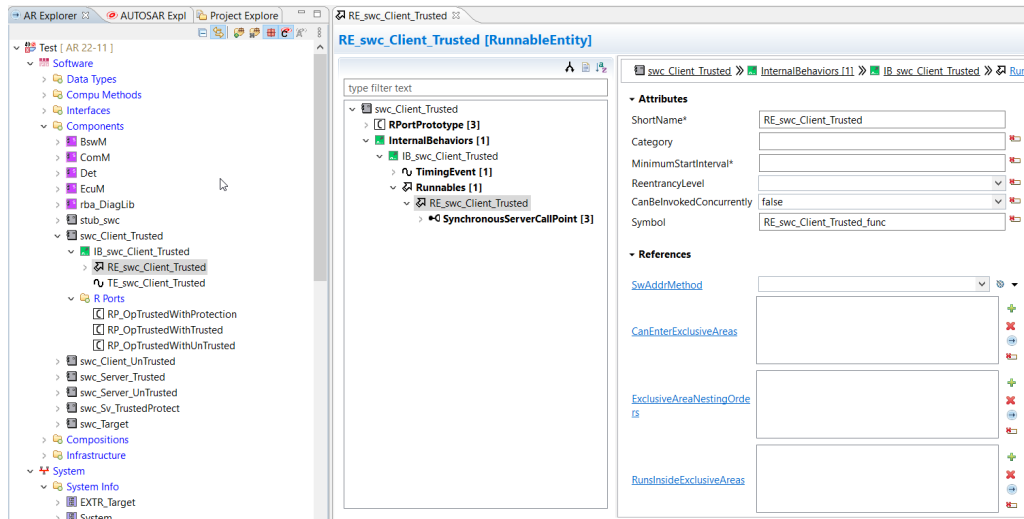


Figure 5.6.Application SWC

5.9.9.2 ECU Extraction

From ECU extraction for the FlatMap, a Flat Instance Descriptor is required to be added for client-server ports when working with Rte_Rips_CSSafety.

- Add a "FlatInstanceDescriptor" by right-clicking **New Child** on Flatmap.
- Configure **AssociatedRtePlugin** field as "CSSafety".
- Choose the **Target** interface for which the plugin shall be applied.

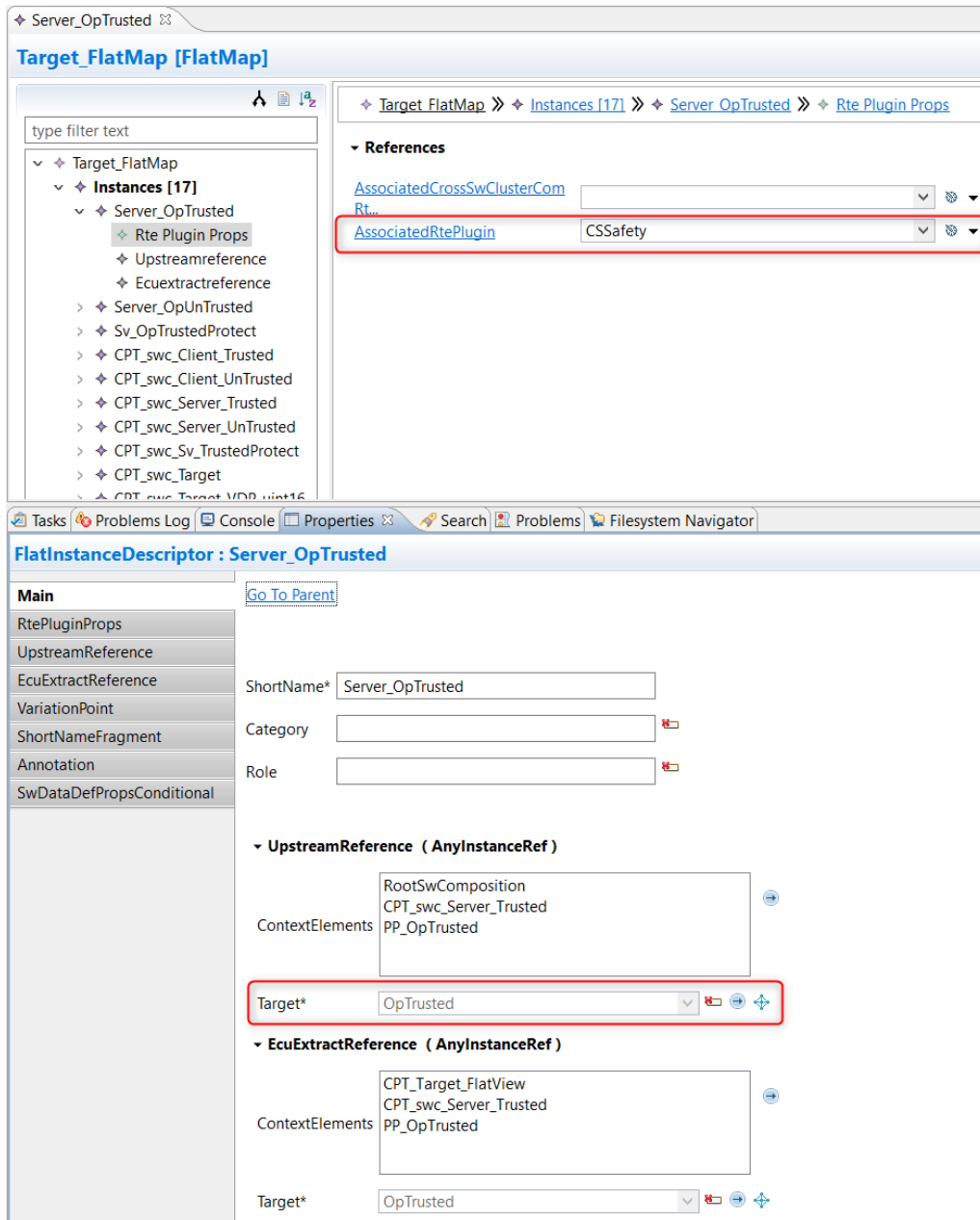


Figure 5.7.Flat Instance Descriptor

5.9.9.3 Create EcucPartition

In the **ECU Navigator**, from **Bsw Modules > Other Modules > EcuC > EcucPartitionCollection**, create new EcucPartitions as shown below, if the partition hosts BSW SWCs, set **EcucPartitionBswModuleExecution** to true.

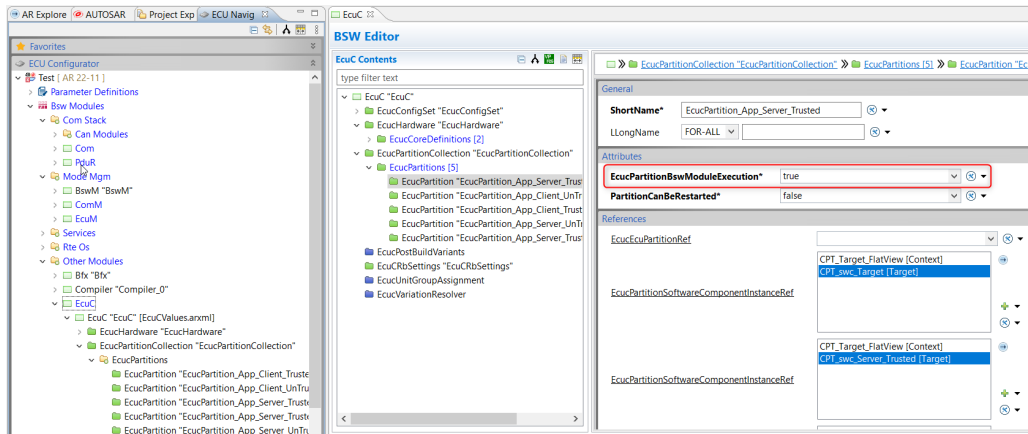


Figure 5.8. Ecuc Partition Configuration

5.9.9.4 Create OsApplication

In the **ECU Navigator**, from **Rte Os > Os**, create new **OsApplication** as shown below, and each **OsApplication** must have an **OsAppEcucPartitionRef**.

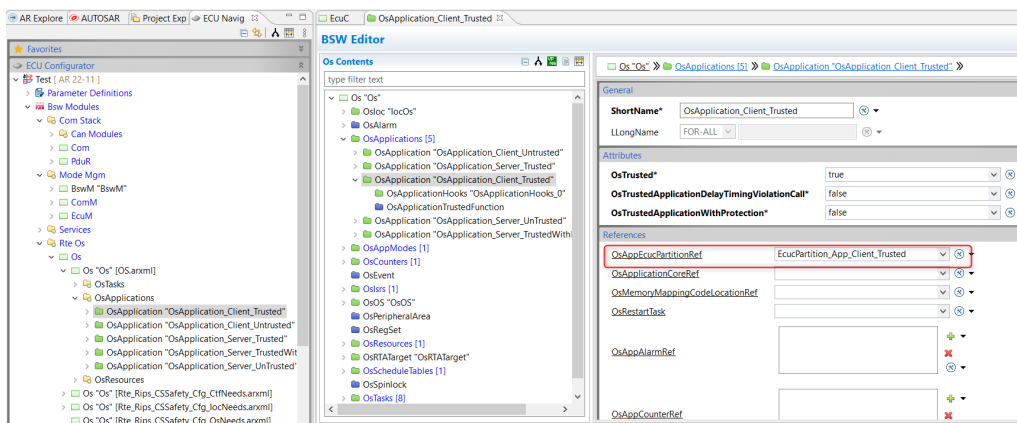


Figure 5.9. OsApplication Configuration

5.9.9.5 Mapping SWC into EcucPartition

From the **EcucPartition** container, at **EcucPartitionSoftwareComponentInstanceRef** field, choose **Configure InstanceRef** to refer to the target SWC.

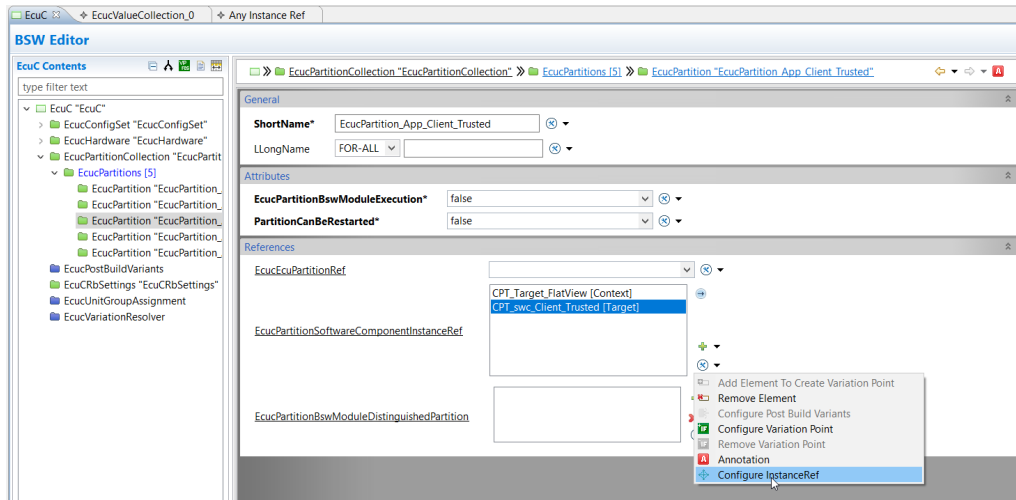


Figure 5.10.Mapping SWC to EcucPartition

5.9.9.6 RTE configuration

Set RIPS ON in the RTE configuration.

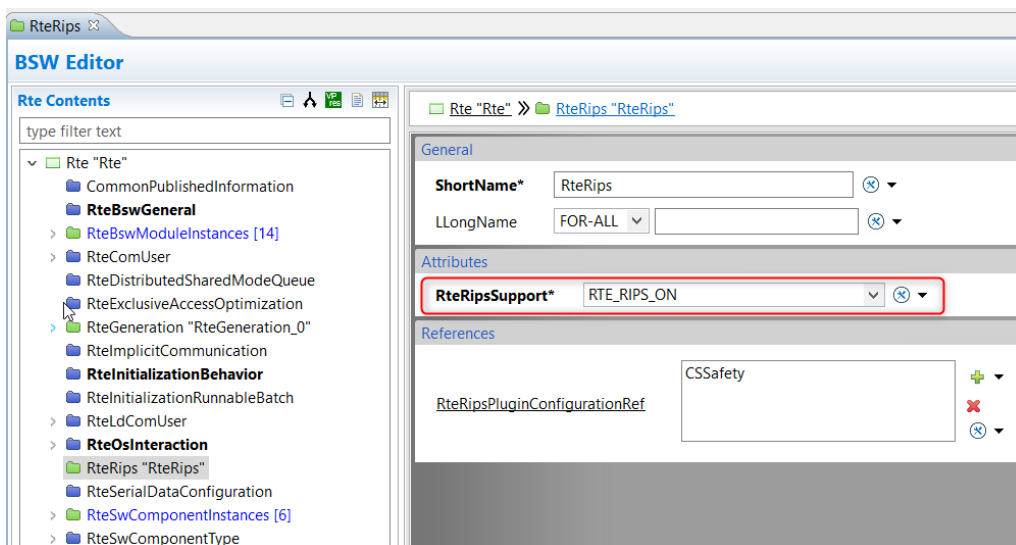


Figure 5.11.Set RIPS ON

5.9.10 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rte_Rips_CSSafety module does not use any exclusive OS resources.

5.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

6 Glossary

C/S

Client Server

CSXfrm

C/S Communication with Transformers

RIPS

RTE Implementation Plugins

Rte_Rips_CSXfrm

RTA-BSW module (RIPS for C/S Communication with Transformers), enables SW-Cs allocated on a specific ECU instance to request services (via SomelpXf, Com and LdCom) from other SW-Cs allocated on a different ECU instance

Rte_Rips_SRCom

RTA-BSW module (RIPS for S/R Communication with Transformers), enables SW-Cs allocated on a specific ECU instance to transmit/receive data via Transformers (SomelpXf, E2EXf) and LdCom to/from other SW-Cs allocated on a different ECU instance

S/R Sender/Receiver**SRCom**

S/R Communication with Transformers

SWC

Software Component